

**A WEB BASED STEGANALYSIS FOR DETECTING AND EXTRACTING
HIDDEN DATA FROM IMAGE FILES**

**MUHAMMAD AMEER BIN IDRIS
52215124748**

**UNIVERSITI KUALA LUMPUR
OCTOBER 2025**

A WEB BASED STEGANALYSIS FOR DETECTING AND EXTRACTING
HIDDEN DATA FROM IMAGE FILES

MUHAMMAD AMEER BIN IDRIS

52215124748

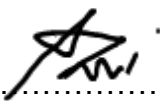
Report Submitted to Fulfill the Partial Requirements
For the Bachelor of Technology (Hons.) in Computer System Security
University Kuala Lumpur

OCTOBER 2026

DECLARATION

I declare that this report is my original work and all references have been cited adequately as required by the University.

Date : 20/06/2026

Signature.....

Full Name: MUHAMMAD AMEER BIN IDRIS

ID Number: 52215124748

APPROVAL PAGE

We have supervised and examined this report and verify that it meets the program and University's requirements for the bachelor's in technology (Hons.) in Computer System Security

Date : _____ Signature.....
Supervisor: Ts. Norhaiza Binti Ya Abdullah
Official Stamp:
(Optional)

Date : _____

Signature.....

Assessor:

Official Stamp:
(Optional)

ACKNOWLEDGEMENT

First and foremost, I would like to express my special thanks of gratitude to my supervisor, Mdm. Norhaiza as well as Final Year Project coordinator, Madam Mardiana Mahari for giving me the opportunity to do this project, which also helped me in doing a lot of research and I came to learn about so many new things on how to do the proper report, and other requirement needed for final year project. Secondly, I would also like to extend my gratitude to my family for their endlessly support and encouragement which is an important source of my strength to complete throughout this project. This project also will not complete without the championship, suggestion, encouragement and support from my classmates and friends who helped me a lot in finalizing this project within the limited time frame.

TABLE OF CONTENTS

CONTENTS

DECLARATION	iii
APPROVAL PAGE	iv
ACKNOWLEDGEMENT	v
TABLE OF CONTENTS	vi
TABLE OF FIGURES	viii
ABSTRACT	x
ABSTRAK	xi
CHAPTER 1: INTRODUCTION	1
1.0 Introduction	1
1.1 Background of Project	1
1.2 Problem Statement	2
1.3 Objectives of Project	3
1.4 Scope and Limitations	3
1.4.1 Scope of the Project	3
1.4.2 Limitations	4
1.5 Significant Of The Project	5
CHAPTER 2: LITERATURE REVIEW	7
2.0 Introduction	7
2.1 Digital Multimedia	7
2.1.2 Common Features of Multimedia Relevant to Steganography	8
2.2 Steganography	9
2.2.1 Definition of Steganography	9
2.2.2 Types of Steganography Techniques	9
2.3 Steganalysis	11
2.3.1 Definition	11
2.3.2 Ways to Detect Hidden Data in Files	11
2.3.3 Forensic Objectives: Detection vs. Extraction	12
2.3.4 Limitations of Existing Steganalysis Tools	12
2.4 Research Gaps	13
CHAPTER 3: METHODOLOGY	14
3.1 Introduction	14

3.2 V-Model Methodology	14
3.2.1 Rational for Selection	14
3.3 Verification Phases (System Design)	15
3.3.1 Requirement Gathering and Analysis	15
3.3.2 Software Design: Architecture Diagram	17
3.3.3 Module Design (System Logic)	18
3.4.1 Prototype	24
CHAPTER 4: SYSTEM DEVELOPMENT	30
4.1 INTRODUCTION.....	30
4.2 System Requirements	30
4.2.1 Hardware Requirements	30
4.2.2 software requirements.....	31
4.3 Preparation	32
4.3.1 Installation Steps	32
4.4 Development.....	35
4.4.1 Core Detection Engine.....	35
4.4.2 Data Extraction Engine & Persistent Analysis	42
4.4.3 Security Hardening, Admin Panel & File Carving	47
4.5 Chapter Summary.....	51
CHAPTER 5: TESTING AND RESULT	52
5.1 INTRODUCTION	52
5.2 UNIT TESTING	52
5.2.1 Functionality Testing (Unit Testing)	52
5.2.2 Integration Testing	54
5.2.3 Security Testing	56
5.2.4 User Acceptance Testing.....	60
5.3 Chapter Summary.....	64
CHAPTER 6: CONCLUSIONS	65
6.1 Introduction	65
6.2 Conclusion of the Overall Project	65
6.3 Discussion of Findings	65
6.3 Achievement of Objectives	66
6.4 Limitations of the Project.....	67
6.5 Recommendations	67
REFERENCES	68
APPENDIX	69

TABLE OF FIGURES

Figure 1: V-Model	15
Figure 2: Gantt Chart	16
Figure 3: Architecture Diagram.....	17
Figure 4: Flowchart of File Upload and Analysis	18
Figure 5: Usecase Diagram	20
Figure 6: ERD Diagram	22
Figure 7: Main Page	24
Figure 8: Account Registration Page	25
Figure 9: Login Page.....	26
Figure 10: Dashboard Page	27
Figure 11: Upload Malicious Image Page	28
Figure 12: Result Scanning Image Page	29
Figure 13: Personal Laptop	30
Figure 14: Active Enviroment.....	33
Figure 15: Required Packages	33
Figure 16: Configure Environment Variables	34
Figure 17: Project Directory Structure.....	34
Figure 18: Application Factory Pattern.....	35
Figure 20 : LSB Detector pt1.....	37
Figure 22: Statistical Analyzer pt1	39
Figure 25: ML Detector pt1	41
Figure 28: Database Models pt1	42
Figure 29: Database Models pt2.....	43
Figure 30: EOF Detection	44
Figure 31: LSB Bit Extraction	45
Figure 32:The Unified Analysis Pipeline pt1	46
Figure 34: Account Lockout & Brute-Force Protection	47
Figure 35: login route checks lockout status	47
Figure 36: Security Header.....	48
Figure 37: File Carving	49
Figure 38: Content Aware Threat Scoring.....	50
Figure 39: Auto Migration at Startup	50
Figure 40: Admin Default Account Seeding.....	51
Figure 41: Occupation of Respondents	61
Figure 42: Domain Familiarity	62
Figure 43: Prior Experience	62
Figure 44: Usability & Interface	63
Figure 45: Performance & Functionality.....	63

LIST OF TABLE

Table 1: V-Model Phase Attribute	14
Table 2: System Requirement	16
Table 3: Hardware Requirement.....	16
Table 4: Hardware	31
Table 5: Software	31
Table 6: Module 1	36
Table 7: Security.....	48
Table 8: Functionality Testing	52
Table 9: Integration Testing	54
Table 10: Brute Force Password Attack Testing	56
Table 11: Upload Malware Testing.....	57
Table 12: Noise Injection Testing	58
Table 13: Inject Corrupted Payload Testing	59

ABSTRACT

Cybercriminals are increasingly using steganography to hide malware in images. Investigating multimedia files is essential for courts but presents a challenge. Tools for Steganalysis are predominantly concerned about detection of binaries. They cannot extract the covert payloads. To fix this serious blind spot, this project introduces safety. A web-based system is made to detect and extract hidden. data captured from image files like PNG, JPG or BMP Created Using V Model. The system employs a multi-engine detection method and three-tier architecture. It can identify hidden messages using the least significant bit (LSB) and other techniques. Methods and an unsupervised machine Learning (Isolation Forest) algorithm to correctly ascertain anomalies in steganography. Most importantly, it has a strong. An extraction engine designed to directly carve out embedded payloads, including text files and executables, from suspect images. The testing further confirmed the system able to detect concealed data even with noisy evasion. Forensic analysis should not involve the recovery of original files. System breakdown. In the end, it offers a digital triage tool that is easy to use and very reliable. Forensic investigators that give shape to the threat to the existence of life or property evidence retrieval.

ABSTRAK

Peningkatan penggunaan steganografi oleh penjenayah siber untuk menyembunyikan data berniat jahat di dalam fail multimedia digital memberikan cabaran forensik yang ketara, memandangkan alat steganalisis sedia ada memberi tumpuan terutamanya kepada pengesanan binari tanpa keupayaan untuk mengekstrak muatan tersembunyi. Bagi menangani titik buta yang kritikal ini, projek ini memperkenalkan sebuah sistem steganalisis berasaskan web yang selamat, direka bentuk untuk mengesan dan mengekstrak maklumat yang tersembunyi daripada fail imej (PNG, JPG, BMP). Dibangunkan menggunakan metodologi V-Model dan seni bina tiga peringkat, sistem ini menggunakan strategi pengesanan pelbagai enjin yang menggabungkan analisis Bit Paling Kurang Signifikan (LSB), kaedah statistik klasik, dan algoritma Pembelajaran Mesin tidak diselia (Isolation Forest) untuk mengenal pasti anomali steganografi dengan tepat. Lebih penting lagi, ia menggabungkan enjin pengestrakan kukuh yang mampu melakukan pengukiran (carving) muatan terbenam seperti fail teks dan fail boleh laku secara terus daripada imej yang disyaki. Ujian komprehensif mengesahkan keberkesanan sistem dalam mengenal pasti data tersembunyi walaupun berhadapan dengan teknik pengelakan suntikan hingar (noise-injected evasion techniques) dan berjaya mendapatkan semula fail asal untuk analisis forensik tanpa sebarang kegagalan sistem. Kesimpulannya, sistem ini menyediakan alat triaj yang mudah diakses dan sangat boleh dipercayai untuk penyiasat forensik digital, seterusnya merapatkan jurang antara sekadar pengesanan ancaman dan pemulihan bukti yang nyata.

CHAPTER 1: INTRODUCTION

1.0 Introduction

Information security has emerged as a critical component of technology in today digital environment. Cybercriminals are always coming up with new ways to send or hide data without being discovered. Steganography which conceals hidden information within common multimedia files including photos, audio, and video is one of the most advanced methods utilized for this purpose.

While steganography has legitimate applications, such copyright protection and digital watermarking, it also presents serious risks when used maliciously. Steganalysis is used to find and retrieve hidden data in order to counter these attacks. This project investigates and creates a web-based steganalysis system that can recognize and retrieve hidden data from multimedia files.

1.1 Background of Project

Steganography is a way of hiding information inside media. This has become a problem for people who work on keeping the internet safe. At first steganography was used for things like keeping conversations private and protecting copyrights. Now bad people use it to hide bad things. Unlike cyberattacks steganography works quietly by putting bad data inside good files like pictures, which makes it hard to find with regular security checks.

In the few years there have been many instances where attackers have hidden bad software, secret instructions or private messages inside files that seem harmless, like videos and pictures. For example, reports from 2021 to 2024 show that bad people used something called Operation Stegano, where they hid code in advertisement pictures to avoid being caught by antivirus software. Also some bad software and robots use pictures to send messages and avoid being detected by security systems. These cases show that regular security tools, like antivirus software and systems that detect intruders mostly look for known software and do not check multimedia files very much.

This creates a problem for people who investigate cybercrimes especially when it comes to illegal conversations, stolen data or spying. Since people usually trust files that have pictures or videos they might not look closely at them. They might miss secret information when they are looking at evidence or responding to a cyberattack. As a result investigators might miss clues or not realize how bad an attack is.

There is a need for a system that can detect suspicious activity, in multimedia files and find the secret information that is hidden inside them. We need to fix this problem to help people who investigate cybercrimes to make the internet safer and to stop people from using steganography to attack us. Steganography is a problem and we need to find a way to stop it. Steganography is being used more and more in cyberattacks and we need to do something about it.

1.2 Problem Statement

The field of steganalysis research is always getting better. There are still some big problems that have not been solved especially when it comes to digital forensics. We can see this in real-life cyber attacks where people are using steganography more and more to hide things like malicious instructions, stolen data and secret communication channels inside multimedia files. For example during Operation Stegano from 2016 to 2018 we saw how attackers were able to hide JavaScript code inside advertisement images, which allowed malware to get past antivirus software during a big cyber-espionage campaign. Recently from 2022 to 2024 cybersecurity reports showed that ransomware groups like WastedLocker and Zloader used image files with hidden payloads to avoid being detected during investigations.

These incidents show us that there are still big problems that have not been solved:

1. Steganalysis Tools Focus on Detection but Not Extraction

Most of the tools we have now can tell us if a file might have steganographic content but they often cannot get the hidden payload out put it back together or decode it. This is a problem for forensic investigators because they need the actual hidden data to prove that someone did something wrong or to figure out what happened during an attack.

2. Ineffective Payload Recovery During Investigations

when we detect hidden content our current methods are not very good at getting the embedded data especially when attackers use their own algorithms or change the structure of the multimedia file. In cases this means we might not get all the evidence we need which can slow down the investigation or make it hard to come to a conclusion.

3. Multimedia Files Are a Security Blind Spot

Most of the time our cybersecurity tools do not look very closely at multimedia files they just do a basic scan to see if they can find anything bad. This gives attackers an opportunity to hide data or malware in image formats, like PNG or JPG. For the teams that investigate cases this means they might miss hidden communication, stolen data or malware and that can make it hard to get all the evidence they need.

1.3 Objectives of Project

Based on the problem statement, this project aims to achieve the following specific objectives:

1. To study techniques used to detect and extract hidden data in multimedia files using steganalysis.
2. To develop a web application that is able to analyse multimedia files and detect the presence of hidden data.
3. To test the application accuracy in detecting and extracting hidden data.

1.4 Scope and Limitations

1.4.1 Scope of the Project

The establishment of a safe and useful web-based steganalysis program is the main goal of this project. The scope of the investigation comprises:

1. Platform: This system is a web-based forensic analysis tool that you can use with any browser. We chose a web platform because it works on lots of devices is easy to use with forensic tools and is safe and secure on a server.

2. Target Users: The people who will use this system are forensic investigators, cybersecurity analysts, law enforcement personnel and researchers who study steganography and cyber threats. These steganalysis users will be the ones using the system.

3. Functionality: The system does a few things. It lets you upload and analyze multimedia files to see if they have been manipulated with steganography. It can detect data and extract secret information from supported files. The system is designed to help steganalysis users find things in files.

4. File Types Supported: For now the system only works with image files like PNG, JPG and BMP. We chose images because they are often used in cybercrime cases that involve steganography. We might add video analysis later but that is not part of this project. The steganalysis program will focus on image files.

5. Security Compliance: We made the system follow the OWASP Application Security Guidelines. This means it is protected against things like SQL Injection, Cross-Site Scripting and other common web vulnerabilities. The steganalysis system is designed to be safe and secure, for steganalysis users.

1.4.2 Limitations

Despite its capabilities, the project has several limitations:

- Limited to image-based steganography: The current prototype does not support audio and video steganalysis.

- Not compatible with all steganographic techniques: If someone uses a very complicated or secret method to hide a message the system might not be able to find it.
- Performance dependent on file characteristics: If the image is very clear or very fuzzy or if it has been compressed a lot the system might not be able to find the hidden message well.
- Unsupported file formats: If an image file is not the type or if it is damaged the system might not be able to look at it or find the hidden message.
- Does not address other forms of steganography: It does not look for hidden messages, in internet traffic, text or computer hardware because that is not what it was made to do.

1.5 Significant Of The Project

The development of a web-based steganalysis prototype is very important for cybersecurity and digital forensics. Cybercriminals are using steganography to hide things in harmless files. This project fixes some problems in current security systems.

The significance of this project is categorized into three primary areas:

1. Bridging the Gap Between Detection and Extraction

- Current Limitation: Existing tools and antivirus scanners often focus solely on detection or signature-based scanning, leaving forensic investigators aware that a file is suspicious but unable to access the hidden content.
- Project Impact: This system will help investigators get the hidden data. This could be programs, stolen login information or secret messages. This is very important for proving someone did something and figuring out how they did it.

2. Eliminating Forensic Blind Spots

- Addressing the Threat: Attackers exploit this trust to smuggle ransomware and command instructions via images like PNG and JPG without triggering alarms.

- Project Impact: This system will look closely at these "blind spots" to make sure hidden threats are found before they can hurt a network or slow down an investigation.

3. Enhancing Accessibility for Law Enforcement and Analysts

- Operational Utility: This system is, on the web so it can be used by people like law enforcement and cybersecurity analysts no matter where they are or what device they use.
- Usability: The system is easy to use so people can quickly. Analyze evidence, which helps speed up the first part of digital investigations. This is the steganalysis prototype. It will really help the cybersecurity and digital forensics field. The steganalysis prototype is a deal because it will make it easier to catch cybercriminals who are using steganography.

CHAPTER 2: LITERATURE REVIEW

2.0 Introduction

This chapter looks at the ideas and theories about steganography, digital multimedia and steganalysis. The main goals of this part are to give people the knowledge they need to understand how secret information is hidden in multimedia files. We also want to explain how different steganalysis algorithms are created to find and get that information. We start by talking about the basics of multimedia. Then we look at steganography and the different ways it is used. After that we focus on research about traditional and modern steganalysis algorithms. Finally, this part of the book points out the gaps in research, on steganography and steganalysis and explains how this project will try to fill those gaps in steganography and steganalysis research.

2.1 Digital Multimedia

2.1.1 Definition and Evolution

Digital multimedia refers to content that combines multiple forms of media such as images, audio, text, and video in digital format. Between 2020 and 2025, researchers identified multimedia files as popular carriers for hidden communication because they contain complex structures that can be slightly modified without noticeable changes (Subramanian et al., 2021). Recent studies emphasize that the high-dimensional nature of modern multimedia provides a "perceptual cover" where secret data is interleaved with the carrier's natural noise component (Subramanian et al., 2021).

In digital forensic literature, multimedia is categorized into several main types based on their specific technical structures:

- Image formats: JPEG, PNG, and BMP are the most commonly used in steganographic research due to their large pixel matrices and high embedding capacity (Subramanian et al., 2021).
- Audio formats: WAV and MP3 are frequently examined because they contain multiple frequency bands suitable for embedding (Subramanian et al., 2021).

- Video files: Formats such as MP4 offer even greater capacity since they include thousands of frames where hidden messages can be distributed (Subramanian et al., 2021).

2.1.2 Common Features of Multimedia Relevant to Steganography

Previous studies highlight three key features of multimedia that support steganography:

1. **Redundancy:** Multimedia files typically contain a significant amount of excess or redundant information that is not strictly required for human perception. This redundancy allows steganographic algorithms to replace or modify certain data bits, such as the least significant bits (LSB) of a pixel, without causing visible distortion (Subramanian et al., 2021). In modern high-resolution files, this "digital noise" provides an extensive workspace for interleaving secret data with the carrier natural data components.
2. **Human Perception Limits:** The human visual and auditory systems have physical limitations, often failing to detect minor fluctuations in colour intensity, brightness, or sound frequency. Steganography exploits these "perceptual blind spots" by embedding information at a level below the human threshold of awareness (Subramanian et al., 2021). Because these subtle changes appear as natural variations or artifacts to a human observer, the existence of a hidden message remains unsuspecting.
3. **Large Data Volume:** Modern multimedia files, particularly high-definition images and videos, possess large file sizes and complex data structures. High-resolution images provide millions of pixels, while video files offer thousands of individual frames, creating a vast "embedding capacity" (Subramanian et al., 2021). This high capacity allows attackers to hide substantial payloads ranging from encrypted text to malicious executable code while keeping the ratio of hidden data to cover data low enough to avoid triggering statistical anomalies.

2.2 Steganography

2.2.1 Definition of Steganography

Steganography is the scientific process and art of hiding information within a digital cover medium so that the very existence of the hidden message remains concealed from observers. Unlike cryptography, which focuses on making the content of a message unreadable through encryption, steganography aims to ensure that the communication remains entirely undetectable (Subramanian et al., 2021).

In a digital context, this involves embedding a secret payload into a carrier file, such as an image or audio clip, resulting in a "stego-object" that appears identical to the original file to the human eye. Modern research emphasizes that the primary goal is to maintain "perceptual transparency," where the modified file does not trigger any suspicion during transmission (Subramanian et al., 2021).

2.2.2 Types of Steganography Techniques

Research identifies several categories of steganographic methods:

a. **Spatial Domain Techniques:** These methods involve the direct modification of the raw data within a file, such as the pixel values of an image. The most widely discussed technique is the Least Significant Bit (LSB) method, which replaces the lowest bits of a pixel with secret data bits. For example, in an 8-bit image, changing the LSB has a negligible effect on the colour, making it invisible to the human eye, though it remains vulnerable to statistical analysis. Sahu & Swain (2020) demonstrate that modern spatial domain enhancements, such as edge-based selection, can hide data in "busy" areas of an image where pixel values change rapidly, making the modification even harder to detect statistically.

b. **Transform Domain Techniques:** These techniques embed data into transformed frequency coefficients, such as those produced by the Discrete Cosine Transform (DCT) or Discrete Wavelet Transform (DWT). Instead of changing pixels directly, the algorithm modifies the mathematical representation of the image, which is then converted back into a viewable format. Subramanian et al. (2021) emphasize that

transform domain techniques offer significantly greater robustness against common image processing like cropping, filtering, and lossy compression.

c. Adaptive Steganography: Also known as content-aware steganography, these approaches vary the embedding strength based on the specific characteristics of the cover file, such as texture, noise, or edge complexity. The algorithm actively avoids "smooth" areas of a file, like a clear blue sky, and instead embeds data in "noisy" regions where changes are naturally masked by the complexity of the image. Li et al. (2022) highlight that adaptive strategies can effectively minimize the "detectable artifacts" left in the cover image by choosing the most difficult to analyze pixels for data storage.

d. Deep Learning-Based Steganography: (Modern Trend) The latest trend in research involves using Neural Networks, specifically Generative Adversarial Networks (GANs), to create steganographic carriers. Kaur & Kocher (2023) document that GAN-based steganography creates highly sophisticated carriers that mimic the statistical properties of natural images perfectly. In this approach, a "Generator" learns to hide data so well that a "Discriminator" (an AI steganalysis tool) cannot tell the difference between a clean file and a hidden-data file.

2.3 Steganalysis

2.3.1 Definition

Steganalysis is formally defined as the scientific process of discovering, detecting, or extracting hidden messages within digital data carriers. While steganography focuses on concealment, steganalysis serves as the essential counter-mechanism used to breach these secure channels to reveal invisible communication. Research consistently distinguishes between two primary forensic objectives: detection, which is the binary determination of whether a secret payload exists within a file, and extraction, which involves the technical retrieval and reconstruction of the hidden data. Bin et al. (2024) refine this definition for the modern era, characterizing steganalysis as a critical forensic necessity. In digital investigations, it is no longer enough to simply know a file is suspicious; steganalysis is required for reconstructing attack chains, identifying threat actors, and providing the concrete evidence needed to prove criminal intent in a court of law.

2.3.2 Ways to Detect Hidden Data in Files

Effective steganalysis requires multiple layers of inspection to identify the presence of unauthorized data:

1. **Statistical Anomaly Analysis:** Traditional steganalysis relies heavily on statistical analysis to find anomalies in the carrier file. This includes Histogram Comparison, which looks for unnatural "flattening" of pixel distributions, as well as noise distribution analysis and pixel correlation testing (Sahu & Swain, 2020).
2. **Signature-Based Scanning:** Much like traditional antivirus software, these methods search for specific patterns or headers left by common steganography tools like Steghide or OpenStego (Alshumrani et al., 2023).
3. **Feature-Based Deep Learning:** As steganographic embedding becomes more sophisticated, modern steganalysis has shifted toward identifying subtle statistical disturbances (Kaur & Kocher, 2023). Convolutional Neural Networks (CNNs) are trained on millions of images to recognize the subtle "digital

signature" left by different algorithms, even when those signatures are invisible to traditional statistical tests (Kaur & Kocher, 2023).

4. Content-Adaptive Detection: Modern AI models are specifically designed to analyze the texture and edge complexity of a file to detect if data has been hidden in "noisy" areas like grass or sand, which are typically used by adaptive steganography to evade detection (Li et al., 2022).

2.3.3 Forensic Objectives: Detection vs. Extraction

In the field of digital forensics, steganalysis is categorized into two distinct primary objectives: detection and extraction (Bin et al., 2024). While these processes are related, they serve different roles in a criminal investigation or security audit.

Detection is defined as the binary determination of whether a secret payload exists within a multimedia file. This stage acts as a "red flag" for investigators, identifying which files have been tampered with and which are clean. However, literature suggests that many current tools stop at this stage, which prevents investigators from knowing what the actual hidden message contains (Bin et al., 2024).

Extraction, on the other hand, involves the technical retrieval and reconstruction of the hidden data. This is a much more complex task as it requires the system to reverse the specific algorithm used by the attacker. From a legal perspective, extraction is a critical forensic necessity because it provides the concrete evidence such as malicious code, stolen credentials, or private communication needed to prove criminal intent in a court of law (Bin et al., 2024). Without extraction capabilities, investigators face a "forensic blind spot" where they know a crime may have occurred but lack the evidence to prove it.

2.3.4 Limitations of Existing Steganalysis Tools

Literature shows several critical gaps in current tools:

- Focus on Detection Only: Most current tools are designed only to detect the presence of steganography but do not perform the necessary extraction (Bin et al., 2024).

- **Ineffective Payload Recovery:** Even when extraction is attempted, current techniques struggle to retrieve embedded data if attackers use custom algorithms or modified file structures (Bin et al., 2024).
- **Real-World Environmental Variance:** Differences in camera sensors, lighting, and compression levels in real-world evidence can create "noise" that confuses AI models, leading to higher false-positive rates (Kaur & Kocher, 2023).

2.4 Research Gaps

Despite the advances in detection, literature reveals critical gaps:

- **Limited Focus on Extraction:** There is a limited focus on extraction; most work emphasizes detection only, leaving forensic investigators with no evidence to present in court (Bin et al., 2024).
- **Accessibility Gaps:** There is a distinct lack of accessible web-based forensic tools; most advanced solutions exist only as experimental academic scripts that are difficult for law enforcement personnel to integrate into standard workflows (Alshumrani et al., 2023).

CHAPTER 3: METHODOLOGY

3.1 Introduction

This chapter is about the system. How we will make it work. We picked ways of doing things to make sure the Web-based Steganalysis System is made without any problems. Here we talk about how the project will go from start to finish and what kind of computer and software we. How the system will be designed. We will go through all the steps of the project. How we will test it to show how the system can find and get hidden data from pictures and videos.

3.2 V-Model Methodology

The research to learn about steganography and steganalysis techniques that already exist. Based on what we need for the project we chose the V-Model as our method. The V-Model is also known as the Verification and Validation Model.

The V-Model is an extension of the waterfall model and is based on the association of a testing phase for each corresponding development phase. This model is highly suitable for this project as it emphasizes strict validation and verification, which is critical for digital forensic tools to ensure the accuracy of payload extraction.

3.2.1 Rational for Selection

As outlined in the table below, the V-Model provides a balance of quality and structure that is essential for digital forensic tools:

Table 1: V-Model Phase Attribute

Phase Attribute	Description and Benefit
High Quality & Reliability	Ensures hidden data extraction algorithms are verified at every step for bit-level accuracy
Clear Structure	The rigidity of the model makes it easy to manage with specific deliverables for each phase.
Early Defect Detection	Testing planning begins during the requirement stage, significantly reducing bugs in the final application.

3.3 Verification Phases (System Design)

V-Model demonstrates the relationships between each phase of the development life cycle and its associated phase of testing. The horizontal and vertical axes represent time or project completeness (left-to-right) and level of abstraction (coarsest-grain abstraction uppermost).

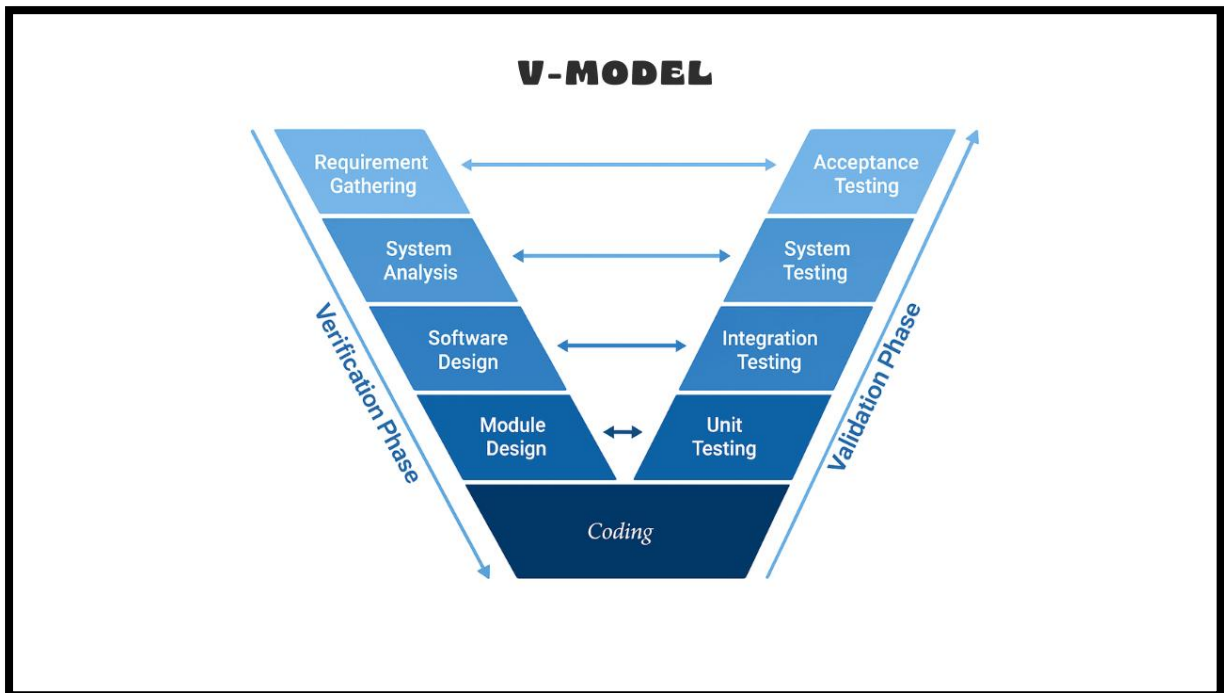


Figure 1: V-Model

3.3.1 Requirement Gathering and Analysis

In this first phase, it is important to understand the project scenario and analyze the requirements for a web-based steganalysis tool. This involves identifying the problem statement (inability to extract payloads), objectives (detect and extract), and scope (image files like PNG, JPG).

Research papers related to steganography algorithms (LSB, DCT) were reviewed. Additionally, project milestones are created to ensure the progress keeps on track.

I. System Requirements:

Before beginning to construct the application, the developer must first confirm that the hardware and software prerequisites are in place. The hardware and software needed to create extensions are listed below.

Table 2: System Requirement

NO	ITEM	PRICE(RM)
1.0	Microsoft Windows 11	FREE
2.0	Visual Studio Code	FREE
3.0	Python	FREE
4.0	Django	FREE
5.0	MySQL	FREE
6.0	HTML5, CSS3, JavaScript	FREE

II. Hardware requirements:

The hardware must fulfill certain performance requirements to facilitate a seamless development process and manage real-time network interception testing. The physical equipment used for the project is listed in Table 2.

Table 3: Hardware Requirement

NO	ITEM	PRICE(RM)
1.0	Personal laptop (ryzen5 Processor Based)	FREE

III. Project Gantt Chart:

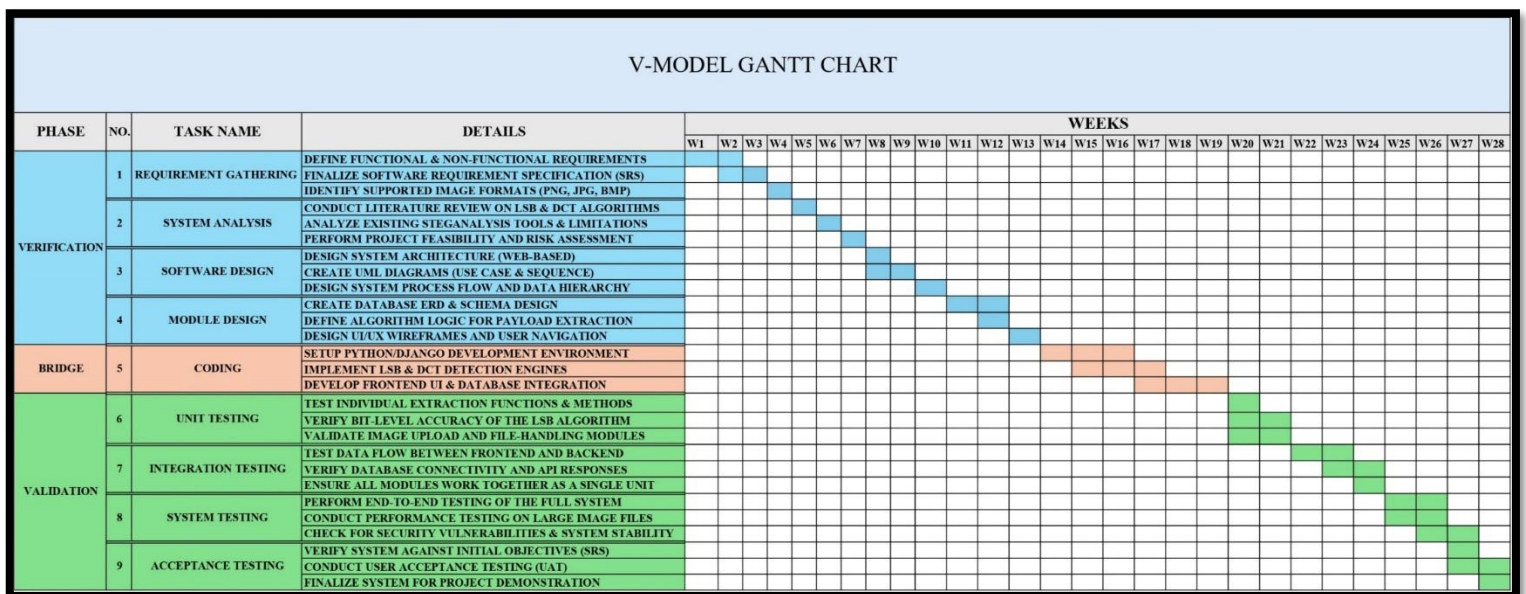


Figure 2: Gantt Chart

3.3.2 Software Design: Architecture Diagram

The architecture diagram illustrates how the user interacts with the web interface and how the backend processes the files.

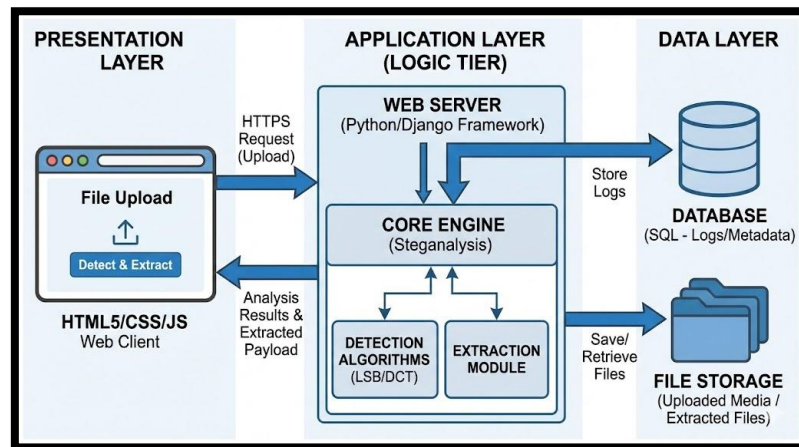


Figure 3: Architecture Diagram

The web-based steganalysis solution is built on a Three-Tier Architecture paradigm. This method divides the Steganalysis solution into three parts: the Presentation, Application and Data layers. This means the user interface, processing logic and data storage are separate making it easier to scale and maintain the Steganalysis solution.

The Presentation Layer is the interface that the forensic analyst uses. This layer is built using HTML5, CSS3 and JavaScript. It provides a web dashboard where users can upload suspicious files securely. The Presentation Layer manages all user interactions checks the inputs and shows the analysis reports so analysts can see the results and download the extracted data.

The Application Layer is the core of the system. It is hosted on a server that uses the Python-based Django Framework. This layer has the Steganalysis Core Engine, which gets the files and runs complex algorithms to analyze them. The engine looks for data by checking the Least Significant Bit and the Discrete Cosine Transform. If it finds data it tries to reverse the process to get the original hidden message.

The Data Layer manages the storage and retrieval of information for the Steganalysis solution. It uses a database to store information, like user activity logs and analysis reports. It also uses a storage system to manage large files and extracted data so the database can perform well without being overloaded with large data assets. The Data Layer is a part of the Steganalysis solution.

3.3.3 Module Design (System Logic)

I. System Flowchart

There are two main flowcharts for this project: the process of file analysis and the process of data extraction.

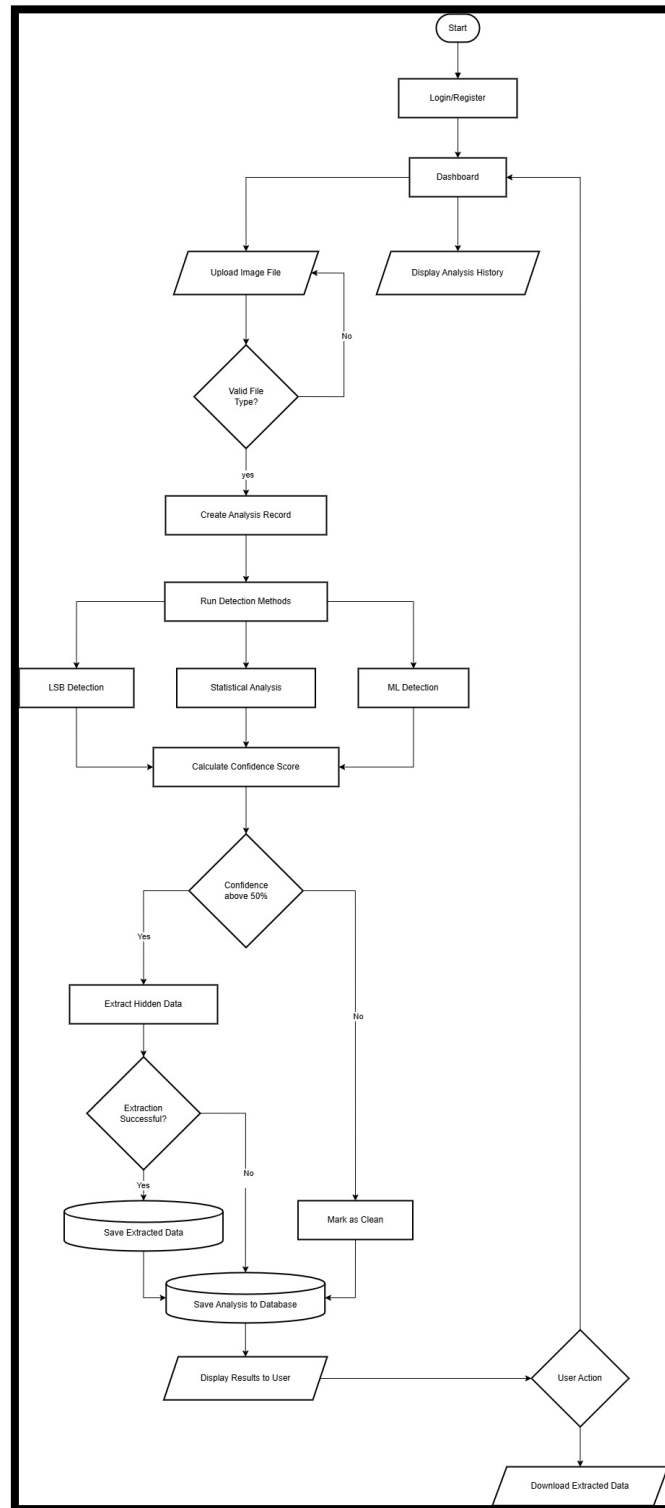


Figure 4: Flowchart of File Upload and Analysis

The web based steganalysis system starts with a place where users have to log in or register first. After that the analyst goes to a page that has all the options. They can look at what they did or start a new investigation. When a user wants to look at a file they have to upload a picture. The system checks if the picture is the type, like PNG, JPG or BMP.

When the file is validated, If the picture is okay the system makes a record of what it does. It uses three ways to check the file: it looks at the bits of information to see if anything is wrong it checks the statistics to see if anything is strange and it uses Machine Learning to look for secret signs. The system puts all these results together to get a score. This score tells us if the picture has anything hidden in it.

If the score is 50% or higher the system says the picture has hidden data and tries to extract it. Whether it works or not the system saves all the results, including any data it found to the database and to a physical storage device. At the end the user gets to see a report and can download any data that was found so they can use it to investigate further or figure out how an attack happened. The web based steganalysis system does all this to help users find data in pictures, like PNG, JPG or BMP files using the Least Significant Bit analysis, statistical analysis and Machine Learning detection.

.

II. Use Case Diagram

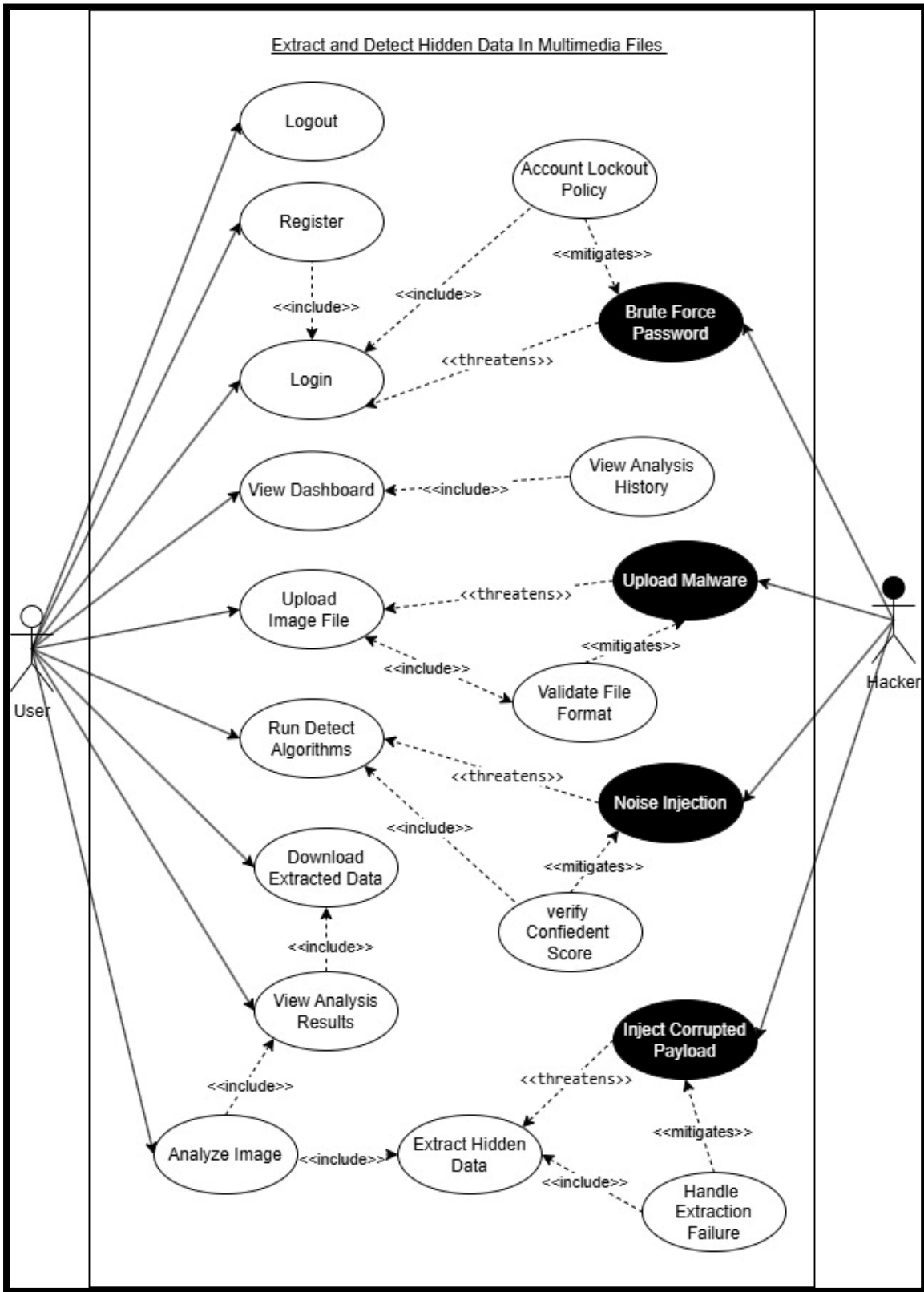


Figure 5: Usecase Diagram

The use case diagram gives an overview of how different people interact with the system and the security measures in place to protect integrity.

1. User Interactions

The person using the system the User Forensic Analyst interacts with several key features:

- Identity Management: This includes Logout, Register and Login. The Login process has an Account Lockout Policy to prevent people from trying to guess passwords with attempts.
- Forensic Workflow: The user can View Dashboard, which includes seeing their Analysis History. The main workflow involves Uploading an Image File, which includes checking if the file format is correct to prevent uploading software.
- Analysis and Results: The user can Run Detect Algorithms, which includes checking the confidence score to prevent results. After detection the user can View Analysis Results. Then Download Extracted Data.

2. Core System Logic (Analyse Image)

The Analyse Image process is the function of the system. It includes Extracting Hidden Data. To keep the system strong the extraction process includes handling failures during extraction to manage cases where a payload might be damaged which helps prevent injecting data.

3. Threat Actor Mitigation

The diagram also shows a Hacker as another actor representing the threats the system is designed to handle. Each threat (shown in black bubbles) is directly addressed by a mitigation process (shown in white bubbles) ensuring the platform is secure and reliable, for digital investigations.

III. ERD Diagram



Figure 6: ERD Diagram

The system is made up of a kind of map that shows how all the data is organized. This map is called the ERD. It uses a structure with three main parts to keep everything safe and trackable.

The data is divided into three groups:

- **User Entity:** This part stores information about each user account. It includes usernames, emails and hashed passwords to keep everything secure. It maintains a one-to-many relationship with the Analysis table, meaning one investigator can conduct multiple forensic sessions.
- **Analysis Entity:** This is the part of the system. It stores a lot of information about each file that is examined. This includes things like the size and type of the file and scores, from detection methods. It also stores the confidence score and tracks if any hidden data was found.
- **Analysis Log Entity:** This part makes sure we have a record of everything that happens in the system. It is connected to the Analysis part. Store every event, timestamp and error message. This allows us to go back and see exactly what happened during an analysis if we need to. The ERD and the Three-Tier relational structure are important for the system to work properly and to maintain integrity and auditability of the data.

3.4 The Coding Phase (Bridge)

The Coding Phase is like a bridge in the V-Model. This is where the ideas and plans from the Verification phases become a software prototype that works. The Coding Phase is very important because it sets up the environment and puts in the main forensic algorithms that make the system work.

- **Development Environment:** The system is built using Visual Studio Code as the tool. It runs on a computer, with Windows 11.
- **Backend Implementation:** The application logic is built using Python and the Django Framework. This includes making the Steganalysis Core Engine. The Steganalysis Core Engine uses the LSB and DCT detection logic.

- **Frontend Development:** The user interface is made using HTML5, CSS3 and JavaScript. This creates a dashboard that's easy to use and works well on different devices. The dashboard is used to upload files and show the results.
- **Data Integration:** A MySQL database is used to handle the information. The file storage system is set up to manage the files and the data that is taken out of them in a way. The Coding Phase is where the system starts to take shape. The Coding Phase is a part of the V-Model and the system would not work without it.

3.4.1 Prototype

I. Main page:

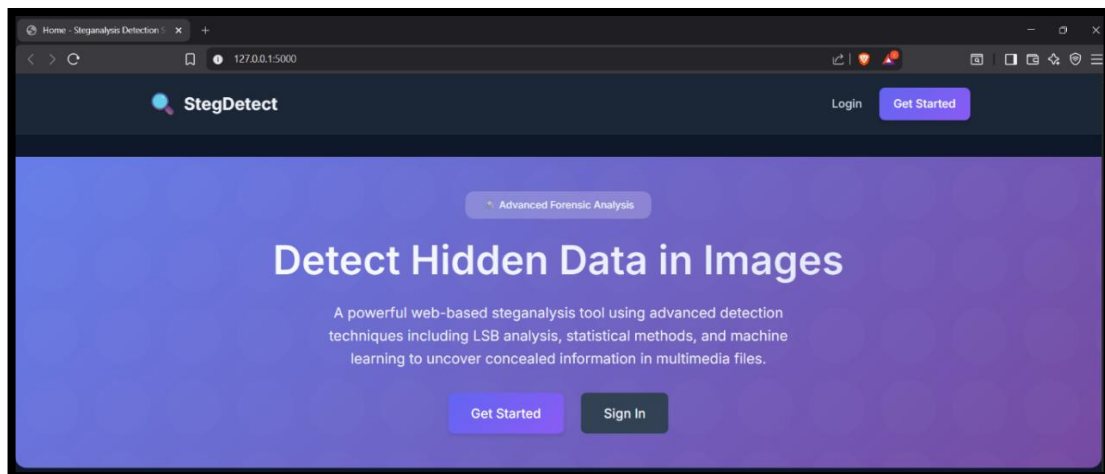


Figure 7: Main Page

The Main Page is where you start with the StegDetect platform. It looks really nice and clean. The StegDetect platform is designed to show you what it can do. The Main Page tells you about the things the StegDetect platform can do like looking really closely at pictures and videos to find hidden information. It uses ways, like LSB analysis and statistical methods and machine learning to find this information. The Main Page has buttons that say "Login" and "Get Started" to help you get to the parts of the StegDetect platform where you can really look at files and figure out what is going on.

II. Account Registration Page:

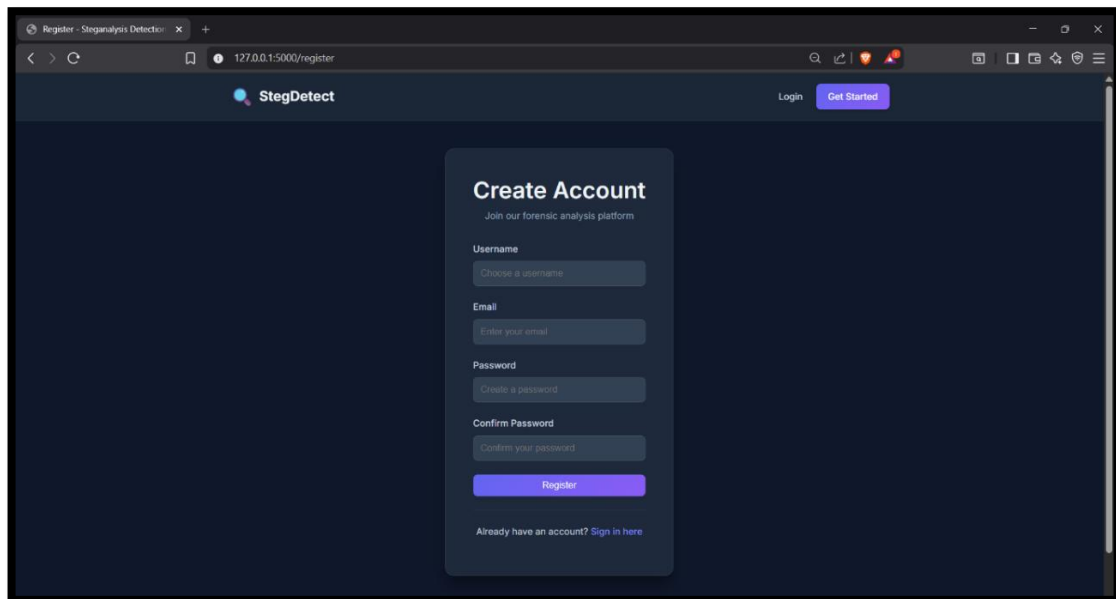
A screenshot of a web browser displaying the 'Create Account' page for StegDetect. The browser's address bar shows '127.0.0.1:5000/register'. The page has a dark blue background. At the top, there is a 'StegDetect' logo and a 'Login' link, with a purple 'Get Started' button. The main content is a white card titled 'Create Account' with the subtitle 'Join our forensic analysis platform'. The card contains four input fields: 'Username' (placeholder: 'Choose a username'), 'Email' (placeholder: 'Enter your email'), 'Password' (placeholder: 'Create a password'), and 'Confirm Password' (placeholder: 'Confirm your password'). Below these fields is a purple 'Register' button. At the bottom of the card, there is a link: 'Already have an account? Sign in here'.

Figure 8: Account Registration Page

The Account Registration Page is where you start to set up a place to work with evidence. To do this you need to give them a username, your email address and a password that is hard to guess. This way each person who uses the system has their name, which is important, for keeping track of what everyone does. The page is easy to use so you do not make mistakes, it even asks you to type your password to make sure it is correct when you sign up for the Account Registration Page. This helps keep the Account Registration Page safe and the Account Registration Page is a part of this process.

III. Login page:

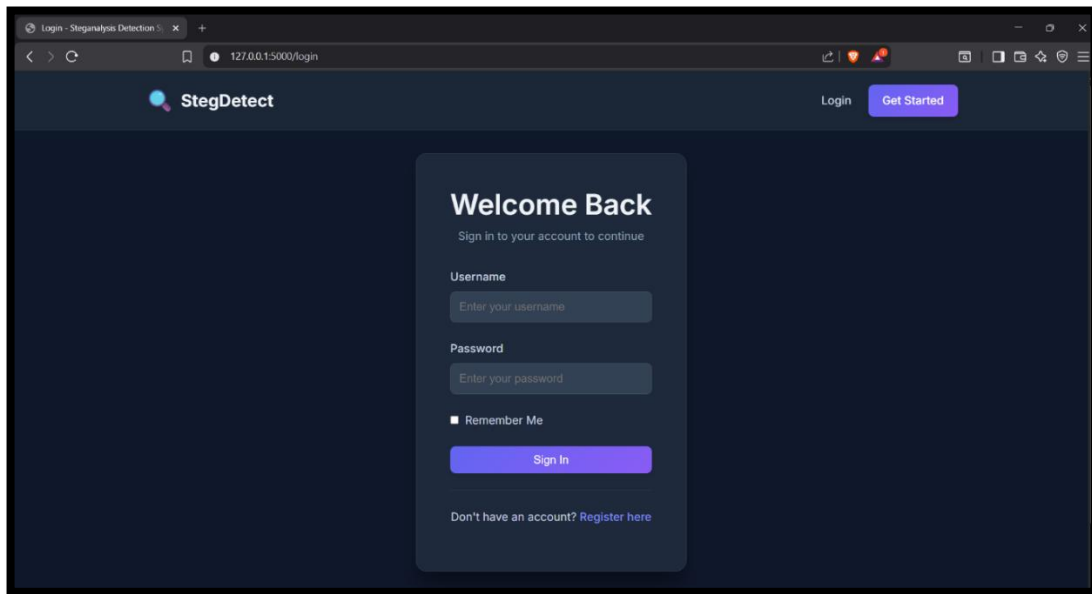


Figure 9: Login Page

The Login Page is where investigators who have used the system before can go to get to the data and analysis they saved. This page has the boxes where you put your username and password. There is also a button that says "Remember Me" which makes it easier to use the system. The Login Page is very important, for keeping people from getting into the private records of investigations. It is the thing that stops people who are not supposed to be looking at these records from getting in. The Login Page is a part of keeping everything safe and secure.

IV. Dashboard page:

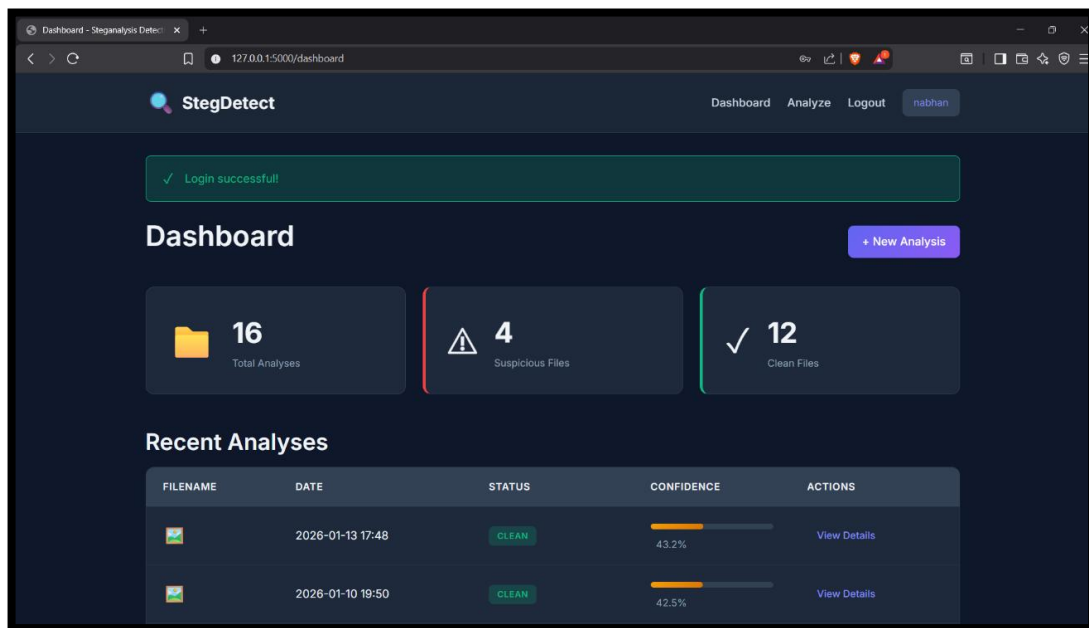


Figure 10: Dashboard Page

The Dashboard Page is like the office for the investigator. It shows you an overview of everything that is going on with the forensic work right away. You can see three things on this page: how many analyses have been done, how many suspicious files have been found and how many clean files have been identified. The Dashboard Page also has a table called "Recent Analyses" below these things. This table shows you a list of files that have been looked at in the order they were examined. For each file you can see when it was looked at what the status is and how confident the system is, in its results. The investigator can use the Dashboard Page to get a sense of what's going on with the forensic activity. The Dashboard Page is helpful for the investigator to see what is happening with the work.

V. Upload malicious image page:

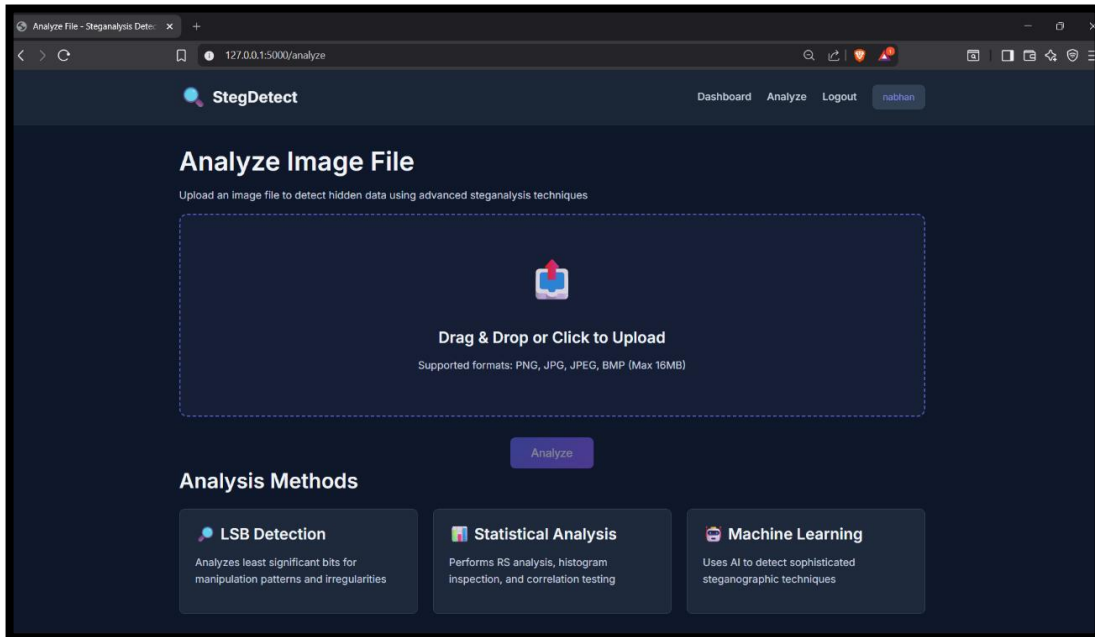


Figure 11: Upload Malicious Image Page

The Analyze Image File page is where the forensic investigation actually starts. This is the place where you can look at the images you need to examine. There is an area in the middle of the page where you can drag and drop your image files. The Analyze Image File page supports a lot of image formats, like PNG, JPG, JPEG and BMP. You can use the Analyze Image File page to look at these types of images. This interface clearly outlines the three analysis methods the system will employ LSB Detection, Statistical Analysis, and Machine Learning so the analyst understands how the file is being scrutinized. The "Analyze" button is positioned at the center to trigger the processing engine once a file is selected.

VI. Result scanning image page:

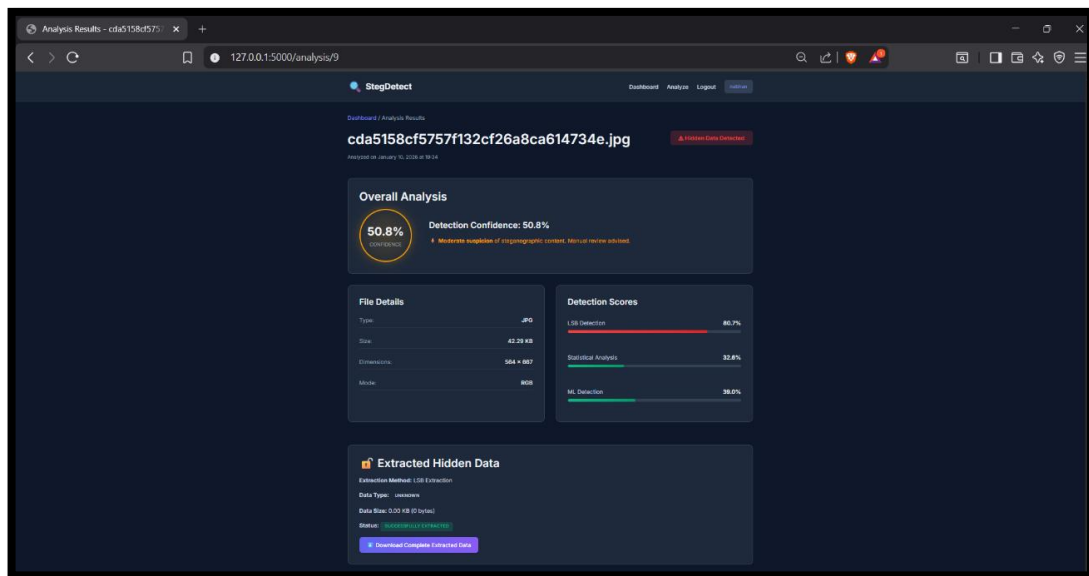


Figure 12: Result Scanning Image Page

The Analysis Results Page provides a comprehensive forensic report for a specific file, headlined by an "Overall Analysis" gauge that shows the final Detection Confidence Score. In this prototype example, the system shows a 50.8% confidence score, labeling it as a "moderate suspicion" of steganographic content. The page further details individual scores for each detection method and provides a section for Extracted Hidden Data, which confirms the extraction status and offers a download button for the recovered payload.

CHAPTER 4: SYSTEM DEVELOPMENT

4.1 INTRODUCTION

This chapter details the development phase of the web-based steganalysis system, StegDetect. Following the "Coding" bridge of the V-Model technique described in Chapter 3, this phase converts the conceptual system design and architectural logic into a workable software prototype. The primary goal of this development cycle is to create a strong Core Engine capable of doing both spatial and frequency-based analysis in order to bridge the current forensic gap between detection and effective payload extraction. This chapter describes the environmental setup, iterative development of the detection algorithms (LSB and DCT), and integration of the three-tier architecture, which includes the Presentation, Application, and Data layers.

4.2 System Requirements

Hardware requirements and software requirements are the first steps that the developer need to ensure it exist before start to develop the application. Below is the hardware and software requirement that required to develop a web based steganalysis detecting and extracting hidden data in image files.

4.2.1 Hardware Requirements

i. Personal Computer



Figure 13: Personal Laptop

Table 4: Hardware

Hardware	Description
Personal laptop	Acer Nitro 5
Features	• AMD Ryzen 5 • 16GB DDR4 • NVIDIA GeForce RTX 3050
Fuction	Program and run the web based system
System Requirement	Windows 10

Figure 13 shows the laptop. Laptop is a device that use for program the system and to run the system. Table 4 shows the minimum specification requirement that need to use for developing the system.

4.2.2 software requirements

Several software tools and libraries were used to develop the Steganalysis project, including Python, Flask, SQLite, and various data analysis libraries. All of these software requirements are explained in detail as follows.

Table 5: Software

No	Software	Function
1	Python 3.8+	Primary programming language used to develop the backend logic and steganography detection algorithms.
2.	Flask	Web framework used for server-side scripting, routing, and creating the web interface.
3.	SQLite	Lightweight database engine used via SQLAlchemy for storing user data, analysis history, and metadata.
4.	VS Code	Integrated Development Environment (IDE) to write, debug, and manage the project's source code.

5.	Pillow (PIL)	Python Imaging Library used for opening, manipulating, and saving image files for analysis.
6.	NumPy & Scikit-learn	Libraries used for numerical computation and implementing statistical analysis (e.g., Chi-square) for detection.
7.	HTML/CSS/JS	Frontend languages are utilized to design the user interface and dashboard.
8.	Git & GitHub	Version control system is used to track changes and manage the project's repository securely.
9.	Web Browser	Used to display the web application, test the interface, and verify security features during development.

4.3 Preparation

4.3.1 Installation Steps

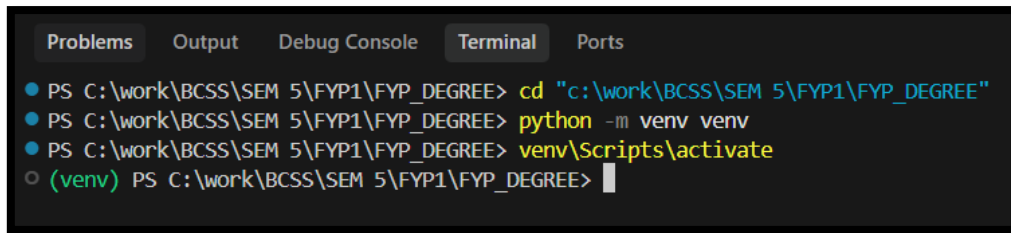
Step 1: Create a Python Virtual Environment

A virtual environment was used to isolate project dependencies from the system Python installation.

```
# Navigate to the project directory
cd "c:\work\BCSS\SEM 5\FYP1\FYP_DEGREE"

# Create virtual environment
python -m venv venv

# Activate virtual environment
venv\Scripts\activate
```



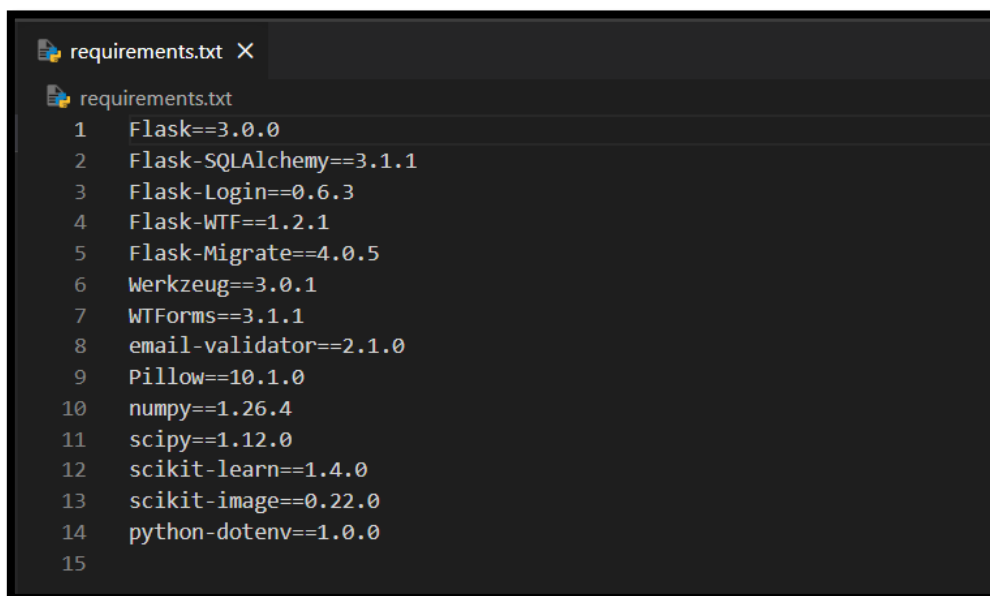
```
Problems Output Debug Console Terminal Ports
PS C:\work\BCSS\SEM 5\FYP1\FYP_DEGREE> cd "c:\work\BCSS\SEM 5\FYP1\FYP_DEGREE"
PS C:\work\BCSS\SEM 5\FYP1\FYP_DEGREE> python -m venv venv
PS C:\work\BCSS\SEM 5\FYP1\FYP_DEGREE> venv\Scripts\activate
(venv) PS C:\work\BCSS\SEM 5\FYP1\FYP_DEGREE>
```

Figure 14: Active Enviroment

Step2: Install Required Packages

All dependencies were captured in requirements.txt and installed with a single command:

```
pip install -r requirements.txt
```

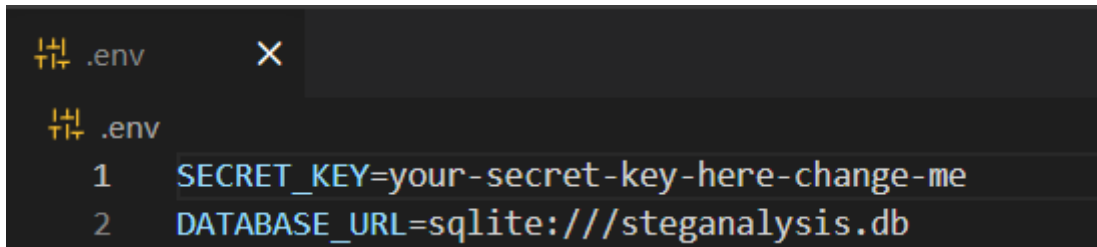


```
requirements.txt X
requirements.txt
1 Flask==3.0.0
2 Flask-SQLAlchemy==3.1.1
3 Flask-Login==0.6.3
4 Flask-WTF==1.2.1
5 Flask-Migrate==4.0.5
6 Werkzeug==3.0.1
7 WTForms==3.1.1
8 email-validator==2.1.0
9 Pillow==10.1.0
10 numpy==1.26.4
11 scipy==1.12.0
12 scikit-learn==1.4.0
13 scikit-image==0.22.0
14 python-dotenv==1.0.0
15
```

Figure 15: Required Packages

Step 3: Configure Environment Variables

A .env file was created to store sensitive configuration such as the secret key and database URI, keeping them out of source control.



```
.env
1 SECRET_KEY=your-secret-key-here-change-me
2 DATABASE_URL=sqlite:///steganalysis.db
```

Figure 16: Configure Environment Variables

Step 4: Project Directory Structure

The project was organized using the Flask Application Factory pattern, separating concerns into distinct modules:

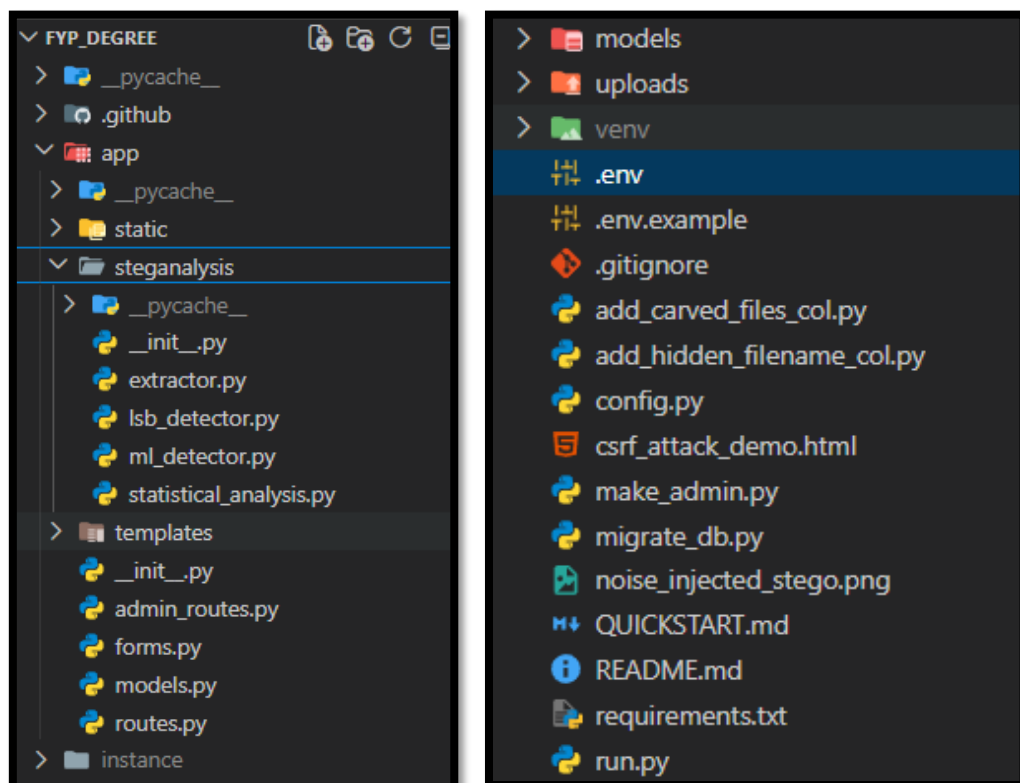


Figure 17: Project Directory Structure

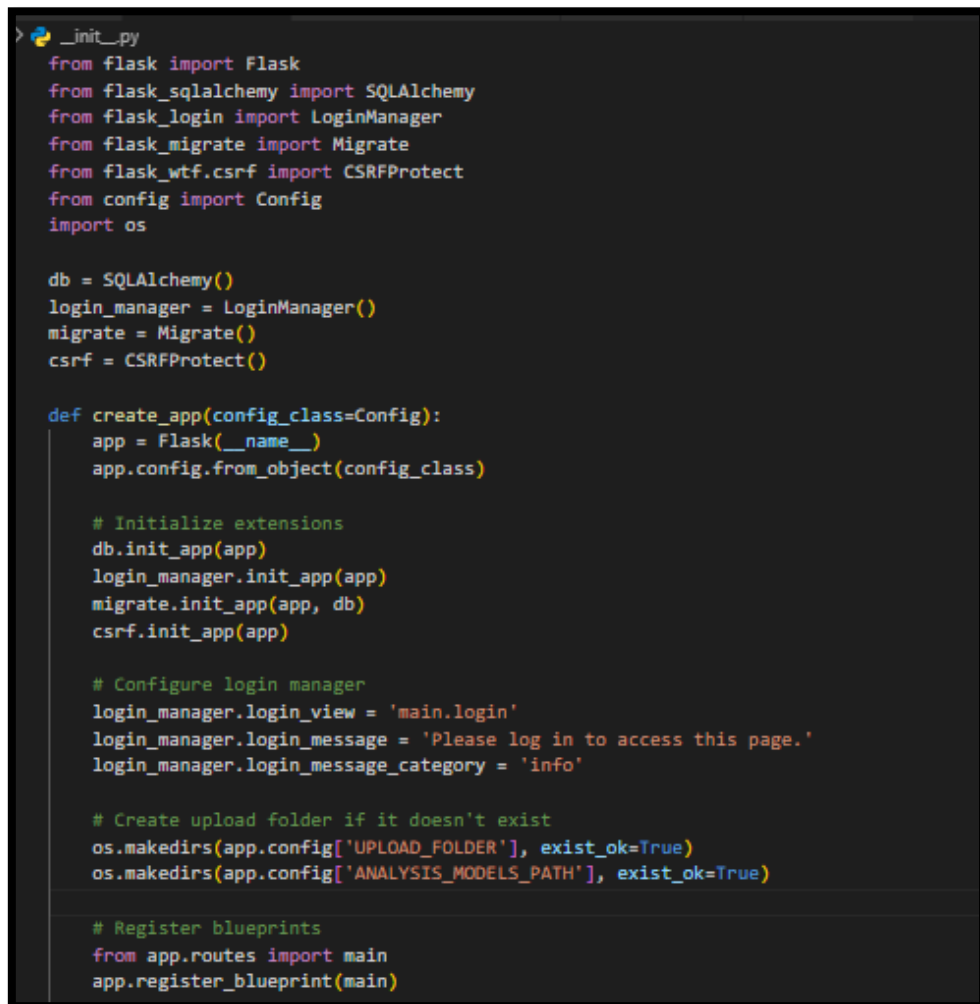
4.4 Development

4.4.1 Core Detection Engine

focused on building the core steganalysis detection engine with three distinct analytical methods. This provided the scientific foundation of the project before any web interface was designed.

1) Application Factory Pattern

The `create_app()` factory function in `app/__init__.py` initializes all Flask extensions and registers blueprints. This pattern allows the application to be easily configured for different environments (development, production, testing).



```
> __init__.py
from flask import Flask
from flask_sqlalchemy import SQLAlchemy
from flask_login import LoginManager
from flask_migrate import Migrate
from flask_wtf.csrf import CSRFProtect
from config import Config
import os

db = SQLAlchemy()
login_manager = LoginManager()
migrate = Migrate()
csrf = CSRFProtect()

def create_app(config_class=Config):
    app = Flask(__name__)
    app.config.from_object(config_class)

    # Initialize extensions
    db.init_app(app)
    login_manager.init_app(app)
    migrate.init_app(app, db)
    csrf.init_app(app)

    # Configure login manager
    login_manager.login_view = 'main.login'
    login_manager.login_message = 'Please log in to access this page.'
    login_manager.login_message_category = 'info'

    # Create upload folder if it doesn't exist
    os.makedirs(app.config['UPLOAD_FOLDER'], exist_ok=True)
    os.makedirs(app.config['ANALYSIS_MODELS_PATH'], exist_ok=True)

    # Register blueprints
    from app.routes import main
    app.register_blueprint(main)
```

Figure 18: Application Factory Pattern

2) Module 1: LSB Detector

The LSB Detector (`app/steganalysis/lsb_detector.py`) analyses the Least Significant Bits of each pixel channel in an image. When data is hidden using LSB steganography, the distribution of LSBs becomes artificially uniform (close to 50% 0s and 50% 1s), and entropy in the LSB plane increases significantly.

Three sub-tests are combined into a weighted score:

Table 6: Module 1

Test	Weight	What it measures
LSB Distribution	30%	Deviation of LSB 1-ratio from expected 0.5
Chi-Square Test	30%	Statistical deviation of LSB pairs
Bit-Plane Entropy	40%	Randomness of the LSB plane

```

class LSBDetector:
    def analyze(self, image_path: str) -> Dict:
        img = Image.open(image_path)
        if img.mode != 'RGB':
            img = img.convert('RGB')
        img_array = np.array(img)

        lsb_distribution = self._check_lsb_distribution(img_array)
        chi_square_lsb = self._chi_square_lsb_test(img_array)
        bit_plane_analysis = self._bit_plane_analysis(img_array)

        score = self._calculate_lsb_score(
            lsb_distribution, chi_square_lsb, bit_plane_analysis
        )
        return {
            'score': score,
            'suspicious': score > 50,
            'lsb_distribution': lsb_distribution,
            'chi_square_result': chi_square_lsb,
            'bit_plane_anomaly': bit_plane_analysis,
            'method': 'LSB Detection'
        }

    def _check_lsb_distribution(self, img_array: np.ndarray) -> float:
        """Deviation from 50/50 LSB distribution signals embedded data."""
        lsb_bits = img_array & 1
        ones_ratio = np.sum(lsb_bits) / lsb_bits.size
        deviation = abs(ones_ratio - 0.5)
        return min(deviation * 500, 100)

    def _chi_square_lsb_test(self, img_array: np.ndarray) -> float:
        """Chi-square uniformity test on the LSB plane."""
        lsb_flat = (img_array & 1).flatten()
        unique, counts = np.unique(lsb_flat, return_counts=True)
        expected = len(lsb_flat) / len(unique)
        chi_square = np.sum((counts - expected) ** 2 / expected)
        return min(chi_square / 10, 100)

    def _bit_plane_analysis(self, img_array: np.ndarray) -> float:
        """High entropy in LSB plane suggests random (hidden) data."""
        def entropy(plane):
            unique, counts = np.unique(plane, return_counts=True)
            probs = counts / counts.sum()
            return -np.sum(probs * np.log2(probs + 1e-10))

        r_ent = entropy(img_array[:, :, 0] & 1)
        g_ent = entropy(img_array[:, :, 1] & 1)
        b_ent = entropy(img_array[:, :, 2] & 1)
        return ((r_ent + g_ent + b_ent) / 3) * 100

    def _calculate_lsb_score(self, dist, chi, bit) -> float:
        return round(dist * 0.3 + chi * 0.3 + bit * 0.4, 2)

```

Figure 19 : LSB Detector pt1

3) Module 2: Statistical Analyzer

The Statistical Analyzer (`app/steganalysis/statistical_analysis.py`) implements three classical steganalysis techniques:

- RS (Regular-Singular) Analysis: Classifies pixel blocks as "Regular" or "Singular" before and after flipping LSBs. In clean images, the ratio of Regular to Singular groups is balanced; LSB embedding distorts this balance.
- Histogram Analysis (Pairs of Values): LSB embedding causes adjacent pixel value pairs (e.g., 100 and 101) to become statistically similar in count. Unusual pair uniformity signals embedding.
- Pixel Correlation Analysis: Natural images have high spatial correlation between adjacent pixels (typically > 0.9). Embedding noise reduces this correlation.

```

class StatisticalAnalyzer:
    def _rs_analysis(self, img_array: np.ndarray) -> float:
        """RS Analysis - detect LSB embedding via Regular/Singular block counts."""
        def flip_lsb(block): return block ^ 1
        def discriminator(block): return np.sum(np.abs(np.diff(block.flatten())))

        gray = np.mean(img_array, axis=2).astype(np.uint8)
        block_size = 4
        h, w = gray.shape
        regular_m, singular_m = 0, 0

        for i in range(0, h - block_size, block_size):
            for j in range(0, w - block_size, block_size):
                block = gray[i:i+block_size, j:j+block_size]
                f = discriminator(block)
                f_m = discriminator(flip_lsb(block))
                if f_m > f: regular_m += 1
                elif f_m < f: singular_m += 1

        total = regular_m + singular_m
        if total == 0: return 0
        rs_difference = abs(regular_m/total - singular_m/total)
        return min(rs_difference * 200, 100)

    def _histogram_analysis(self, img_array: np.ndarray) -> float:
        """Detect Pairs of Values (PoV) anomaly in pixel histograms."""
        anomaly_scores = []
        for channel in range(3):
            hist, _ = np.histogram(img_array[:, :, channel], bins=256, range=(0,256))
            pair_diffs = [abs(hist[i] - hist[i+1]) for i in range(0, 255, 2)]
            mean_d = np.mean(pair_diffs)
            std_d = np.std(pair_diffs)
            cv = std_d / (mean_d + 1e-10)
            anomaly_scores.append(cv)
        return min(np.mean(anomaly_scores) * 20, 100)

    def _pixel_correlation_analysis(self, img_array: np.ndarray) -> float:
        """Reduced spatial correlation signals LSB manipulation."""
        correlations = []
        for ch in range(3):
            c = img_array[:, :, ch]
            h_corr = np.corrcoef(c[:, :-1].flatten(), c[:, 1:].flatten())[0, 1]
            v_corr = np.corrcoef(c[:, :-1].flatten(), c[1:, :].flatten())[0, 1]
            correlations.append((h_corr + v_corr) / 2)
        loss = 0.95 - np.mean(correlations)
        return min(loss * 500, 100) if loss > 0 else 0

```

Figure 20: Statistical Analyzer pt1

4) Module 3: ML Detector

The ML Detector (`app/steganalysis/ml_detector.py`) uses scikit-learn's Isolation Forest, an unsupervised anomaly detection algorithm. Rather than requiring labelled training data, it extracts a 39-dimensional feature vector from the image and evaluates whether the image's statistical signature is anomalous compared to an expected "normal" profile.

Feature Vector (39 features):

- Channel-wise statistics: mean, std, median, Q25, Q75 ($\times 3$ channels = 15 features)
- LSB plane mean and std ($\times 3$ channels = 6 features)
- Horizontal and vertical pixel differences mean/std ($\times 3$ channels = 12 features)
- Per-channel Shannon entropy (3 features)
- Bit-plane entropy for bits 0–2 (3 features)

```

from sklearn.ensemble import IsolationForest

class MLDetector:
    def __init__(self):
        self.model = IsolationForest(
            contamination=0.1, # Expected outlier proportion
            random_state=42,
            n_estimators=100
        )

    def _extract_features(self, img_array: np.ndarray) -> np.ndarray:
        features = []
        # 1. Channel statistics
        for ch in range(3):
            cd = img_array[:, :, ch]
            features.extend([np.mean(cd), np.std(cd), np.median(cd),
                             np.percentile(cd, 25), np.percentile(cd, 75)])

        # 2. LSB plane statistics
        for ch in range(3):
            lsb = img_array[:, :, ch] & 1
            features.extend([np.mean(lsb), np.std(lsb)])

        # 3. Pixel difference statistics
        for ch in range(3):
            cd = img_array[:, :, ch]
            features.extend([np.mean(np.diff(cd,axis=1)), np.std(np.diff(cd,axis=1)),
                             np.mean(np.diff(cd,axis=0)), np.std(np.diff(cd,axis=0))])

        # 4. Shannon entropy per channel
        for ch in range(3):
            hist, _ = np.histogram(img_array[:, :, ch], bins=256, range=(0,256))
            hist = hist / hist.sum()
            features.append(-np.sum(hist * np.log2(hist + 1e-10)))

        # 5. Bit-plane entropy
        for bit in range(3):
            bp = (img_array[:, :, 0] >> bit) & 1
            _, cnts = np.unique(bp, return_counts=True)
            probs = cnts / cnts.sum()
            features.append(-np.sum(probs * np.log2(probs + 1e-10)))

        return np.array(features)

    def _analyze_features(self, features: np.ndarray) -> float:
        heuristic = self._heuristic_analysis(features) # Rule-based: 60% weight
        anomaly = self._anomaly_detection(features) # IF model: 40% weight
        return round(heuristic * 0.6 + anomaly * 0.4, 2)

```

Figure 21: ML Detector pt1

4.4.2 Data Extraction Engine & Persistent Analysis

1) Database Models

- User Model: Stores registered user accounts with password hashing, login tracking, and account lockout fields.

```
class User(UserMixin, db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(64), unique=True, nullable=False, index=True)
    email = db.Column(db.String(120), unique=True, nullable=False, index=True)
    password_hash = db.Column(db.String(256), nullable=False)
    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    last_login = db.Column(db.DateTime)
    is_active = db.Column(db.Boolean, default=True)
    failed_login_attempts = db.Column(db.Integer, default=0, nullable=False)
    locked_until = db.Column(db.DateTime, nullable=True)
    is_admin = db.Column(db.Boolean, default=False, nullable=False)
    analyses = db.relationship('Analysis', backref='user', lazy='dynamic',
                              cascade='all, delete-orphan')

    MAX_FAILED_ATTEMPTS = 5
    LOCKOUT_MINUTES = 15

    def set_password(self, password):
        self.password_hash = generate_password_hash(password)

    def check_password(self, password):
        return check_password_hash(self.password_hash, password)

    def is_locked_out(self) -> bool:
        if self.locked_until is None: return False
        return datetime.now(timezone.utc).replace(tzinfo=None) < self.locked_until

    def record_failed_login(self):
        self.failed_login_attempts = (self.failed_login_attempts or 0) + 1
        if self.failed_login_attempts >= self.MAX_FAILED_ATTEMPTS:
            self.locked_until = datetime.utcnow() + timedelta(minutes=self.LOCKOUT_MINUTES)
```

Figure 22: Database Models pt1

- Analysis Model: Stores a complete record of each image analysis, including all detection scores, extracted data metadata, and file-carving results.

```
class Analysis(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey('user.id'))
    filename = db.Column(db.String(255), nullable=False)
    original_filename = db.Column(db.String(255), nullable=False)
    file_path = db.Column(db.String(512), nullable=False)
    file_size = db.Column(db.Integer)
    file_type = db.Column(db.String(10))
    analyzed_at = db.Column(db.DateTime, default=datetime.utcnow, index=True)
    has_hidden_data = db.Column(db.Boolean, default=False)
    confidence_score = db.Column(db.Float)
    lsb_detection_score = db.Column(db.Float)
    statistical_score = db.Column(db.Float)
    ml_detection_score = db.Column(db.Float)
    extracted_data_available = db.Column(db.Boolean, default=False)
    extracted_data_path = db.Column(db.String(512))
    extracted_data_preview = db.Column(db.Text)
    extracted_data_type = db.Column(db.String(50))
    extracted_data_size = db.Column(db.Integer)
    extraction_method = db.Column(db.String(50))
    hidden_filename = db.Column(db.String(255))
    carved_files = db.Column(db.Text) # JSON list of carved file descriptors
    image_width = db.Column(db.Integer)
    image_height = db.Column(db.Integer)
    image_mode = db.Column(db.String(20))
    analysis_notes = db.Column(db.Text)
```

Figure 23: Database Models pt2

2) Multi-Strategy Data Extraction Engine

The DataExtractor class (app/steganalysis/extractor.py) attempts to recover hidden payloads using four strategies in priority order:

- Append after EOF Detection: Some steganography methods append data after the image's end-of-file marker (e.g., using the Windows copy /b image.jpg + secret.txt out.jpg command). The extractor reads the raw bytes of the file and looks for data beyond the legitimate image end.

```
extractor.py
app > steganalysis > extractor.py
12 class DataExtractor:
13
14     def _extract_appended_data(self, image_path: str, output_dir: str) -> Dict:
15
16         try:
17             with open(image_path, 'rb') as f:
18                 raw = f.read()
19
20                 appended = None
21                 fmt = None
22
23                 # --- JPEG: find last 0xFF 0xD9 ---
24                 if raw[:3] == b'\xff\xd8\xff':
25                     eoi = raw.rfind(b'\xff\xd9')
26                     if eoi != -1 and eoi + 2 < len(raw):
27                         appended = raw[eoi + 2:]
28                         fmt = 'JPEG'
29
30                 # --- PNG: find IEND chunk end ---
31                 elif raw[:4] == b'\x89PNG':
32                     iend = raw.rfind(b'IEND')
33                     if iend != -1:
34                         after_iend = iend + 8 # 'IEND' (4) + CRC (4)
35                         if after_iend < len(raw):
36                             appended = raw[after_iend:]
37                             fmt = 'PNG'
38
39                 # --- GIF: find 0x3B trailer ---
40                 elif raw[:6] in (b'GIF89a', b'GIF87a'):
41                     trailer = raw.rfind(b'\x3b')
42                     if trailer != -1 and trailer + 1 < len(raw):
43                         appended = raw[trailer + 1:]
44                         fmt = 'GIF'
45
46                 # --- BMP: use declared file size from header ---
47                 elif raw[:2] == b'BM' and len(raw) > 6:
48                     declared_size = int.from_bytes(raw[2:6], 'little')
49                     if declared_size < len(raw):
50                         appended = raw[declared_size:]
51                         fmt = 'BMP'
52
53                 if appended and len(appended) >= 4:
54                     # Strip leading/trailing whitespace / nulls that copy /b adds
55                     appended = appended.strip(b'\x00\r\n ')
56                     if len(appended) < 4:
57                         return {'extracted': False}
58
59                     data_type, ext = self._detect_data_type(appended)
60                     hidden_name = self._detect_hidden_filename(appended, ext)
61                     safe_name = hidden_name.replace('/', '_').replace('\\', '_')
62                     output_path = os.path.join(output_dir, safe_name)
63
64                     with open(output_path, 'wb') as f:
65                         f.write(appended)
66
67                     carved = self._carve_files_from_blob(appended)
68
69                     return {
70                         'extracted': True,
71                         'method': f'Append-after-EOF ({fmt})',
72                         'output_path': output_path,
73                         'data_size': len(appended),
74                         'data_preview': self._get_data_preview(appended),
75                         'data_type': data_type,
76                         'hidden_filename': hidden_name,
77                         'carved_files': carved # list of dicts, may be empty
78                     }
79                 except Exception:
80                     pass
81
82         return {'extracted': False}
```

Figure 24: EOF Detection

- **LSB Bit Extraction:** The extractor reads the LSBs of each pixel channel in row-major order and reassembles the bits into bytes. Three sub-strategies are attempted:
 1. **OpenStego header detection** — checks for the OpSt magic bytes that indicate an OpenStego-format embedding, then reads the structured header to recover the filename and payload.
 2. **Delimiter-based extraction** — reads bytes until the ##### end-of-message marker is encountered.
 3. **Length-header extraction** — reads a 32-bit big-endian length prefix and then extracts that many bytes.

```
def _extract_lsb_bits(self, img_array: np.ndarray) -> np.ndarray:
    """Extract LSBs from all channels in row-major order."""
    lsb_bits = img_array & 1
    height, width, channels = img_array.shape
    bits = []
    for i in range(height):
        for j in range(width):
            for c in range(channels):
                bits.append(lsb_bits[i, j, c])
    return np.array(bits, dtype=np.uint8)

def _extract_until_delimiter(self, bits: np.ndarray) -> Optional[bytes]:
    """Stop reading bits when the '#####' end-of-message marker is found."""
    delimiter_bytes = b'#####'
    max_bytes = min(len(bits) // 8, 512_000)
    buffer = bytearray()
    for i in range(max_bytes):
        byte_bits = bits[i*8 : i*8+8]
        if len(byte_bits) < 8: break
        byte_value = 0
        for bit in byte_bits:
            byte_value = (byte_value << 1) | int(bit)
        buffer.append(byte_value)
        if buffer.endswith(delimiter_bytes):
            payload = bytes(buffer[:-len(delimiter_bytes)])
            return payload if len(payload) > 0 else None
    return None

def _extract_with_length_header(self, bits: np.ndarray) -> Optional[bytes]:
    """Read a 32-bit big-endian length field followed by that many payload bytes."""
    if len(bits) < 32: return None
    message_length = 0
    for bit in bits[:32]:
        message_length = (message_length << 1) | int(bit)
    if message_length < 10 or message_length > 100_000: return None
    total_bits = 32 + message_length * 8
    if len(bits) < total_bits: return None
    return self._bits_to_bytes(bits[32:total_bits], max_bytes=message_length)
```

Figure 25: LSB Bit Extraction

3) The Unified Analysis Pipeline

The `perform_analysis()` function in `app/routes.py` orchestrates the entire detection-to-extraction workflow:

```
def perform_analysis(filepath, original_filename, user_id):
    analysis = Analysis(user_id=user_id, filename=..., ...)

    # Step 1: Run all three detectors
    lsb_result = LSBDetector().analyze(filepath)
    stat_result = StatisticalAnalyzer().analyze(filepath)
    ml_result = MLDetector().analyze(filepath)

    analysis.lsb_detection_score = lsb_result.get('score', 0)
    analysis.statistical_score = stat_result.get('score', 0)
    analysis.ml_detection_score = ml_result.get('score', 0)

    # Step 2: Calculate overall confidence (average of three scores)
    scores = [analysis.lsb_detection_score,
              analysis.statistical_score,
              analysis.ml_detection_score]
    analysis.confidence_score = sum(scores) / len(scores)
    analysis.has_hidden_data = analysis.confidence_score > 50

    # Step 3: Always attempt data extraction
    extract_dir = os.path.join(current_app.config['UPLOAD_FOLDER'], 'extracted')
    os.makedirs(extract_dir, exist_ok=True)
    extraction_result = DataExtractor().extract(filepath, extract_dir)

    if extraction_result.get('extracted'):
        analysis.extracted_data_available = True
        analysis.extracted_data_path = extraction_result.get('output_path')
        analysis.extraction_method = extraction_result.get('method')
        analysis.extracted_data_preview = extraction_result.get('data_preview')
        analysis.extracted_data_type = extraction_result.get('data_type')
        analysis.extracted_data_size = extraction_result.get('data_size')
        analysis.hidden_filename = extraction_result.get('hidden_filename')

        # Store file-carving results as JSON
        carved = extraction_result.get('carved_files') or []
        analysis.carved_files = json.dumps(carved) if carved else None

        # Override confidence with content-aware threat floor
        analysis.has_hidden_data = True
        analysis.confidence_score = max(analysis.confidence_score, threat_floor)

    db.session.add(analysis)
    db.session.commit()
    return {'id': analysis.id, 'success': True}
```

Figure 26: The Unified Analysis Pipeline pt1

4.4.3 Security Hardening, Admin Panel & File Carving

- Account Lockout & Brute-Force Protection: The User model was extended with failed_login_attempts and locked_until fields. After 5 consecutive failed logins, the account is locked for 15 minutes.

```
41 MAX_FAILED_ATTEMPTS = 5
42 LOCKOUT_MINUTES = 15
43
44 def is_locked_out(self) -> bool:
45     """Return True if the account is currently locked."""
46     if self.locked_until is None:
47         return False
48     now = datetime.now(timezone.utc).replace(tzinfo=None) # naive UTC
49     return now < self.locked_until
50
51 def record_failed_login(self):
52     """Increment the failed-login counter and lock the account if threshold reached."""
53     self.failed_login_attempts = (self.failed_login_attempts or 0) + 1
54     if self.failed_login_attempts >= self.MAX_FAILED_ATTEMPTS:
55         from datetime import timedelta
56         self.locked_until = datetime.utcnow() + timedelta(minutes=self.LOCKOUT_MINUTES)
57
58 def reset_failed_logins(self):
59     """Clear the counter and lockout after a successful login."""
60     self.failed_login_attempts = 0
61     self.locked_until = None
```

Figure 27: Account Lockout & Brute-Force Protection

The login route checks lockout status before processing credentials:

```
35
36 | --- Account lockout check ---
37 if user and user.is_locked_out():
38     remaining = int((user.locked_until - datetime.utcnow()).total_seconds() // 60) + 1
39     flash(
40         f'Account locked due to too many failed attempts. '
41         f'Please try again in {remaining} minute(s).',
42         'danger'
43     )
44     return render_template('login.html', form=form)
45
46 if user is None or not user.check_password(form.password.data):
47     # Record the failed attempt (and lock if threshold reached)
48     if user:
49         user.record_failed_login()
50         db.session.commit()
51         if user.is_locked_out():
52             flash(
53                 'Account locked due to too many failed attempts. '
54                 f'Please try again in {user.LOCKOUT_MINUTES} minute(s).',
55                 'danger'
56             )
57             return render_template('login.html', form=form)
58     flash('Invalid username or password', 'danger')
59     return render_template('login.html', form=form)
60
61 # Successful login - reset lockout counter
62 user.reset_failed_logins()
63 login_user(user, remember=form.remember_me.data)
```

Figure 28: login route checks lockout status

- HTTP Security Headers: After every response, security headers are injected to protect against common browser-level attacks:

Table 7: Security

Header	Protection
X-Content-Type-Options: nosniff	Prevents MIME-type sniffing attacks
X-Frame-Options: SAMEORIGIN	Prevents clickjacking via iframes
X-XSS-Protection: 1; mode=block	Activates browser XSS filter
Strict-Transport-Security	Forces HTTPS on subsequent visits

```

45     # Security headers
46     @app.after_request
47     def set_security_headers(response):
48         response.headers['X-Content-Type-Options'] = 'nosniff'
49         response.headers['X-Frame-Options'] = 'SAMEORIGIN'
50         response.headers['X-XSS-Protection'] = '1; mode=block'
51         response.headers['Strict-Transport-Security'] = 'max-age=31536000; includeSubDomains'
52         return response
53 
```

Figure 29: Security Header

- File Carving (Multi-Payload Detection): In reality, an attacker can concatenate multiple files (e.g., a PDF + a ZIP + an EXE) into a single payload. Version 3 added the `_carve_files_from_blob()` method that scans the raw payload for known magic-byte signatures and enumerates each embedded file individually.

Supported Magic Bytes:

```
FILE_SIGNATURES = [
    (b'PK\x03\x04',      '.zip',  'ZIP Archive'),
    (b'Rar!\x1a\x07',    '.rar',  'RAR Archive'),
    (b'7z\xbc\xaf\x27\x1c', '.7z',  '7-Zip Archive'),
    (b'%PDF',            '.pdf',  'PDF Document'),
    (b'MZ',              '.exe',  'Windows Executable (EXE/DLL)'),
    (b'\x7fELF',         '.elf',  'Linux Executable (ELF)'),
    (b'\x89PNG',         '.png',  'PNG Image'),
    (b'\xff\xd8\xff',    '.jpg',  'JPEG Image'),
    # ... (more signatures)
]

def _carve_files_from_blob(self, blob: bytes) -> list:
    """Scan the payload blob for known file signatures (multi-payload detection)."""
    hits = []
    for magic, ext, description in self.FILE_SIGNATURES:
        start = 0
        while True:
            pos = blob.find(magic, start)
            if pos == -1: break
            hits.append((pos, magic, ext, description))
            start = pos + 1

    if not hits: return []

    # Sort and deduplicate by offset (prefer longer magic matches)
    hits.sort(key=lambda h: (h[0], -len(h[1])))
    deduped, last_offset = [], -1
    for pos, magic, ext, description in hits:
        if pos == last_offset: continue
        deduped.append((pos, magic, ext, description))
        last_offset = pos

    if len(deduped) < 2: return [] # Single type + not multi-payload

    # Build carved file descriptors
    carved = []
    for idx, (pos, magic, ext, description) in enumerate(deduped):
        next_pos = deduped[idx+1][0] if idx+1 < len(deduped) else len(blob)
        carved.append({
            'filename': f'carved_{idx+1:02d}_{description.replace(" ", "_")}{ext}',
            'description': description,
            'extension': ext,
            'offset': pos,
            'size': next_pos - pos,
            'magic_hex': ' '.join(f'{b:02X}' for b in magic),
        })
    return carved
```

Figure 30: File Carving

- Content Aware Threat Scoring: payload-type-aware threat floors. The pixel-level detectors (LSB, statistical, ML) cannot judge how dangerous a payload is a ZIP archive and an .exe executable are both suspicious, but the latter poses a significantly higher threat.

The extraction pipeline now adjusts the confidence score based on the type of hidden file detected:

```

333 # Content-aware threat scoring based on extracted payload type
334 # The pixel-level detectors alone can't judge payload danger - we do it here.
335 DANGEROUS_TYPES = {'py', 'exe', 'bat', 'sh', 'ps1', 'vbs', 'js', 'cmd', 'msi', 'dll', 'scr'}
336 SUSPICIOUS_TYPES = {'zip', 'rar', '7z', 'tar', 'gz', 'pdf', 'doc', 'docx', 'xls', 'xlsx'}
337
338 hidden_ext = ''
339 if analysis.hidden_filename:
340     parts = analysis.hidden_filename.rsplit('.', 1)
341     hidden_ext = parts[-1].lower() if len(parts) > 1 else ''
342 elif analysis.extracted_data_type:
343     hidden_ext = analysis.extracted_data_type.lower().strip('.')
344
345 if hidden_ext in DANGEROUS_TYPES:
346     # Executable / script payload - very high threat
347     threat_floor = 90.0
348 elif hidden_ext in SUSPICIOUS_TYPES:
349     # Archive or document - medium-high threat
350     threat_floor = 80.0
351 else:
352     # Unknown payload type - still suspicious
353     threat_floor = 70.0
354
355 analysis.has_hidden_data = True
356 analysis.confidence_score = max(analysis.confidence_score or 0, threat_floor)

```

Figure 31: Content Aware Threat Scoring

- Auto Migration at Startup: To handle incremental schema changes across deployments without requiring manual flask db migrate commands, an `_auto_migrate_db()` function was added to `app/__init__.py`. It inspects the live database schema and adds any missing columns found in the SQLAlchemy models.

```

def _auto_migrate_db():
    """Add missing columns to existing tables on every app start (safe to re-run)."""
    from sqlalchemy import inspect, text

    inspector = inspect(db.engine)
    migrations = {
        'analysis': [
            ('hidden_filename', 'VARCHAR(255)'),
            ('carved_files', 'TEXT'),
        ],
        'user': [
            ('failed_login_attempts', 'INTEGER DEFAULT 0 NOT NULL'),
            ('locked_until', 'DATETIME'),
            ('is_admin', 'BOOLEAN DEFAULT 0 NOT NULL'),
        ],
    }

    with db.engine.connect() as conn:
        for table, columns in migrations.items():
            existing = [c['name'] for c in inspector.get_columns(table)]
            for col_name, col_type in columns:
                if col_name not in existing:
                    conn.execute(text(
                        f'ALTER TABLE {table} ADD COLUMN {col_name} {col_type}'
                    ))
                    conn.commit()
                    print(f'[auto-migrate] Added column: {table}.{col_name}')

```

Figure 32: Auto Migration at Startup

- Admin Default Account Seeding: A `_seed_default_admin()` function ensures that a default admin account is always available, especially useful on fresh deployments or cloud environments where the database is empty.

```
65 def _seed_default_admin():
66     """
67     Ensures that a default admin account exists in the database.
68     This is especially useful for cloud deployments where the SQLite DB is empty.
69     """
70     from app.models import User
71
72     # Check if any admin exists
73     admin_exists = User.query.filter_by(is_admin=True).first()
74     if not admin_exists:
75         try:
76             # Create default admin account
77             default_admin = User(username='admin', email='admin@stegdetect.local')
78             default_admin.set_password('admin123')
79             default_admin.is_admin = True
80             db.session.add(default_admin)
81             db.session.commit()
82             print("[seed] Created default admin account: username='admin', password='admin123'")
83         except Exception as e:
84             db.session.rollback()
85             print(f"[seed] Warning: Failed to create default admin account: {e}")
```

Figure 33: Admin Default Account Seeding

4.5 Chapter Summary

The Analyse Image File page is where we start looking into the details of an investigation. This page has an area where you can drag and drop your image files. It works with image formats like PNG, JPG, JPEG and BMP. This part of the project was about turning our ideas into a website. We used Python and Flask to build the backend and an SQLite database to store user information and analysis results. A big accomplishment was creating the Steganalysis Core Engine. This engine uses three methods to detect hidden information, LSB Detection, Statistical Analysis and Machine Learning. It gives us a confidence score that tells us how sure we are about our findings. We also developed an extraction engine that can find and recover hidden data. This is a deal because it helps us fill in the gaps in forensic research. To keep our application safe we added security features, like account lockout policies, HTTP security headers and content-aware threat scoring. The Analyse Image File page is a part of our system, and it is where the active forensic investigation begins.

CHAPTER 5: TESTING AND RESULT

5.1 INTRODUCTION

This part of the book is about what we found out from trying out the Steganalysis System. We had to try out the Steganalysis System to make sure it works the way it is supposed to. This is an important part of making sure the Steganalysis System is good. We want the Steganalysis System to do what it is supposed to do. We want it to be safe from bad people who might try to hurt it. We also want the people who use the Steganalysis System to like it. We did a few kinds of tests, on the Steganalysis System. We did tests to see if the Steganalysis System works right tests to see if all the parts of the Steganalysis System work together tests to see if the Steganalysis System is safe and tests to see if the people who use the Steganalysis System like it and think it is good.

5.2 UNIT TESTING

5.2.1 Functionality Testing (Unit Testing)

Table 8: Functionality Testing

Test Case	:	1
Project	:	A web based steganalysis detecting and extracting hidden data in image files
Test Case	:	The Functionality of Steganalysis and Data Extraction in the Web Platform
Executing Date	:	May 2026
Description: This testing is to show the functionality of the system, including user authentication, image uploading, steganalysis scoring, and hidden data extraction.		

No	Module	Actual Result	Expected Result	Result (Pass/Fail)
1.	User Registration	User information has been securely recorded in the SQLite database.	The registration is successful and saved in the database.	Pass
2.	User Login	User is authenticated and redirected to the main dashboard.	System grants access only registered users with correct credentials.	Pass
3.	Image Upload	The image is successfully uploaded, and a secure filename is generated and stored in the server directory.	Image is uploaded without errors and restricted to allowed extensions (PNG, JPG, BMP).	Pass
4.	Steganalysis Detection	The system processes the image and returns LSB, Statistical, and Machine Learning confidence scores.	The system accurately detects the probability of hidden data in the uploaded image.	Pass
5.	Data Extraction	The system successfully extracts hidden payloads (e.g., .txt, .zip, .exe) using delimiter and signature validation.	Hidden files are extracted with their original file types, and no garbage binary noise is produced.	Pass

5.2.2 Integration Testing

In this project, the integration between the web application and various client environments (web browsers) is key to making this system successful. Below is the Integration Testing table that has been done for this project.

Table 9: Integration Testing

No	Module	Description	Expected Result	Browser Version (Result)		
				Google Chrome	Mozilla Firefox	Microsoft Edge
1.	System UI Rendering	The system's front-end code successfully integrates with the browser engine.	Able to view the dashboard and interact with the system application smoothly.	Pass	Pass	Pass
2.	File Upload Handling	System can process and upload large image files from the client's local machine to the server.	Upload process is successful without browser timeouts.	Pass	Pass	Pass

3.	Downloading Extracted Data	Server sends extracted binary files back to the user's browser.	User successfully receives the extracted hidden file via browser download manager.	Pass	Pass	Pass
----	----------------------------------	---	---	------	------	------

5.2.3 Security Testing

This testing is the most important testing where developers can know if the built system application is secure enough. This project performed four types of security testing based on the identified threats in the system's Misuse Case Diagram, which are Brute Force Password Attack testing, Upload Malware testing, Noise Injection testing, and Inject Corrupted Payload testing.

5.2.3.1 Brute Force Password Attack Testing

This testing is performed by attempting multiple incorrect password entries on the login page to verify that the Account Lockout Policy is enforced correctly.

Table 10: Brute Force Password Attack Testing

Test Case : 1 Project : A web based steganalysis detecting and extracting hidden data in image files Test Case : Brute Force Password Attack Testing on the Steganalysis Application Executing Date : May 2026					
Description: This testing is to show the security level of the login module against repeated automated password-guessing attempts by a hacker.					
No	Test Case Scenario	Account Accessible (1–4 Attempts)	Account Locked on 5th Attempt	Lockout Message Displayed	Counter Resets on Successful Login
1.	Brute Force Password Attack on the Login module of the Steganalysis System	Pass	Pass	Pass	Pass

5.2.3.2 Upload Malware Testing

This testing is performed by attempting to upload non-image files (e.g., .txt, .py, .exe) through the file upload interface to verify that the file format validation correctly blocks malicious uploads.

Table 11: Upload Malware Testing

Test Case : 2					
Project : A web based steganalysis detecting and extracting hidden data in image files					
Test Case : Upload Malware Testing on the Steganalysis Web Application					
Executing Date : May 2026					
Description: This testing is to show the security level of the file upload module against attempts to upload malicious or non-image files onto the server.					
No	Test Case Scenario	Upload Blocked (Invalid Extension)	Error Message Displayed	File Not Saved to Server	System Remains Stable
1.	Upload a .txt file disguised as an image to the Steganalysis System	Pass	Pass	Pass	Pass
2.	Upload a .py script file to the Steganalysis System	Pass	Pass	Pass	Pass

5.2.3.3 Noise Injection Testing

This testing is performed by uploading a stego-image that has been modified with random pixel noise after embedding, to verify that the detection algorithms can still identify hidden data anomalies.

Table 12: Noise Injection Testing

Test Case : 3				
Project : A web based steganalysis detecting and extracting hidden data in image files				
Test Case : Noise Injection Testing on the Steganalysis Application				
Executing Date : May 2026				
Description: This testing is to show the security level of the steganalysis detection module against noise-injected images intended to evade detection.				
No	Test Case Scenario	Anomaly Detected Despite Noise	Confidence Score Flagged as Suspicious	File Not Saved to Server
1.	Upload a noise-injected stego-image to the Steganalysis System	Pass	Pass	Pass

5.2.3.4 Inject Corrupted Payload Testing

This testing is performed by uploading a stego-image whose hidden payload bytes have been manually corrupted using a Hex Editor, to verify that the extraction failure is handled gracefully without crashing the application.

Table 13: Inject Corrupted Payload Testing

Test Case : 4					
Project : A web based steganalysis detecting and extracting hidden data in image files					
Test Case : Inject Corrupted Payload Testing on the Steganalysis Application					
Executing Date : May 2026					
Description: This testing is to show the security level of the data extraction module against corrupted or malformed hidden payloads that may be used to crash or destabilize the system.					
No	Test Case Scenario	Extraction Attempted	Failure Handled Gracefully	No Application Crash	Error Logged Internally
1.	Upload a stego-image with a manually corrupted hidden payload to the Steganalysis System	Pass	Pass	Pass	Pass

5.2.4 User Acceptance Testing

Objective of UAT: User Acceptance Testing was conducted to evaluate the StegDetect system from the perspective of its intended end-users: digital forensic investigators, cybersecurity analysts, and researchers. Unlike functional or security testing, which verifies technical correctness, UAT assesses the system's usability, practical utility, and overall fit within real-world forensic workflows. The goal was to determine whether the system effectively bridges the gap between detection and extraction as outlined in the problem statement.

Methodology: A structured questionnaire was distributed to a representative sample of 15 respondents with backgrounds in cybersecurity, digital forensics, and information technology. The questionnaire was divided into three sections:

1. Demographics & User Background: To establish the credibility and relevance of the respondents.
2. Usability & Interface: To evaluate the system's ease of use, navigation clarity, and overall user experience.
3. Performance & Functionality: To assess the system's core forensic capabilities, including detection accuracy, extraction reliability, and speed of analysis.

5.2.4.1 SECTION 1: Demographics & User Background

Most of the people who answered were Computer System Security students in their year or they were IT professionals and system analysts. These Computer System Security students and IT professionals and system analysts have the knowledge they need to look at a technical forensic tool. You can see this in the Figures in Section 1. A lot of the users said they were pretty good at cybersecurity and many of them had learned about steganography before which made what they thought important. The Computer System Security students and IT professionals and system analysts gave us feedback on the forensic tool.

1. Occupation of Respondents

The majority of the testing group consisted of Students, representing 80% (12 respondents) of the total participants. IT professionals made up 13% (2 respondents), and Cybersecurity professionals accounted for 7% (1 respondent).

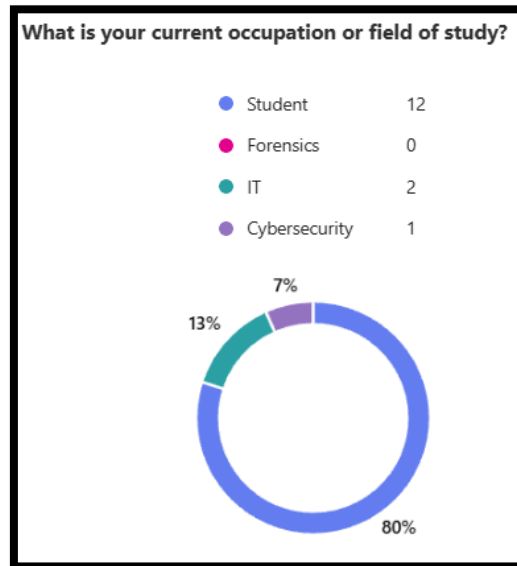


Figure 34: Occupation of Respondents

2. Domain Familiarity

Participants reported the highest familiarity with general Cybersecurity (9 respondents) and Image Processing (6 respondents). A smaller segment was familiar with Steganography/Steganalysis specifically (2 respondents).

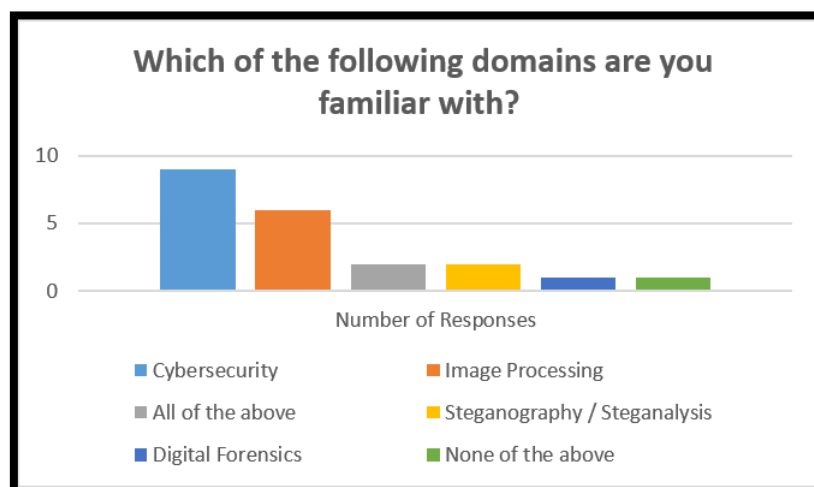


Figure 35: Domain Familiarity

3. Prior Experience

Experience with dedicated steganography or steganalysis software was evenly split among the testing group: 33% (5 respondents) had prior experience, 33% had no prior experience, and 33% were unsure.

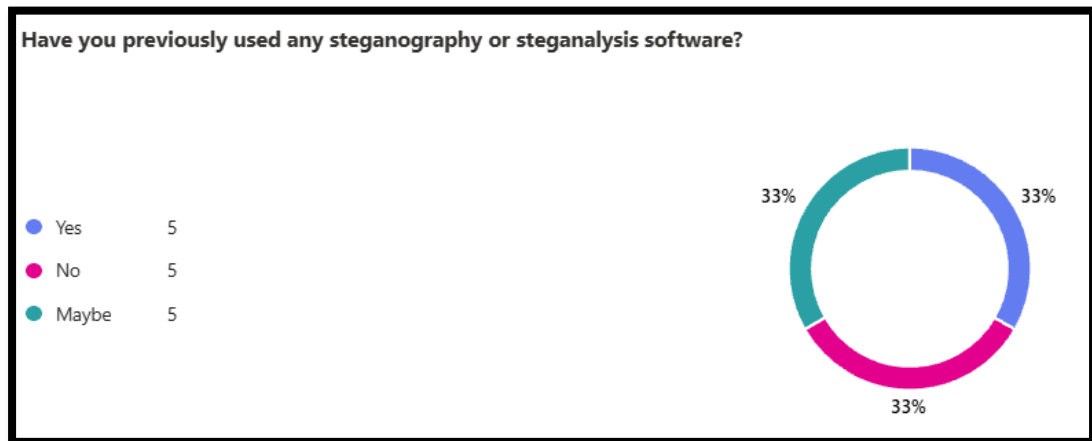


Figure 36: Prior Experience

5.2.4.2 Section 2: Usability & Interface

A clear majority of users (over 70%) rated the system's navigation and interface as "Very Easy" or "Easy" to use. The dashboard layout and the drag-and-drop upload feature were specifically praised as intuitive. The visual presentation of the detection confidence score (gauge meter) and the clear separation of LSB, Statistical, and ML scores were highlighted as excellent for rapid triage during investigations.

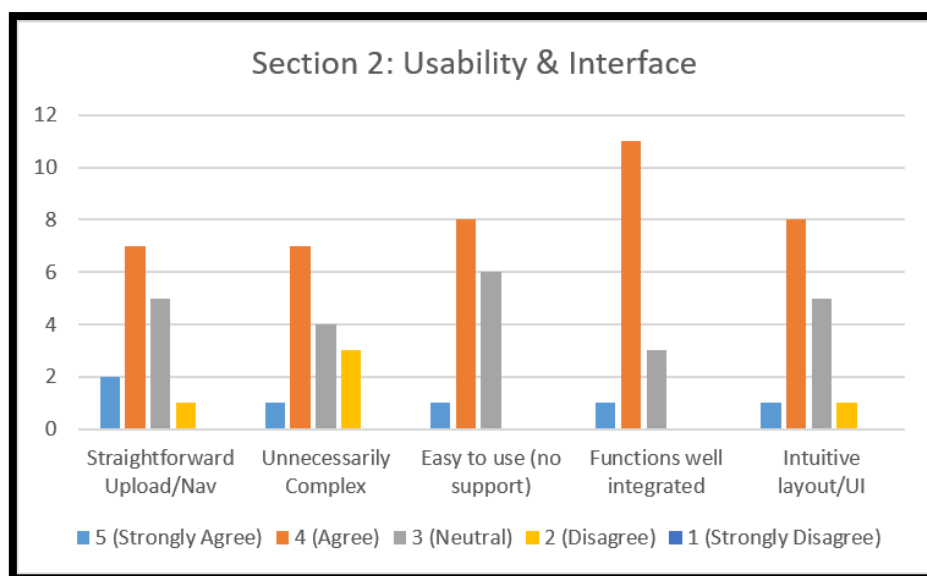


Figure 37: Usability & Interface

5.2.4.3 Section 3: Performance & Functionality

The most critical findings related to the core forensic functions. Over 80% of respondents rated the detection and extraction capabilities as "Effective" or "Highly Effective." Users specifically valued the ability to download extracted payloads (e.g., .txt, .zip files) and the inclusion of file carving, which revealed multiple embedded file types. The speed of analysis for standard image files (under 5MB) was rated as "Fast" or "Very Fast" by most users, indicating acceptable real-world performance.

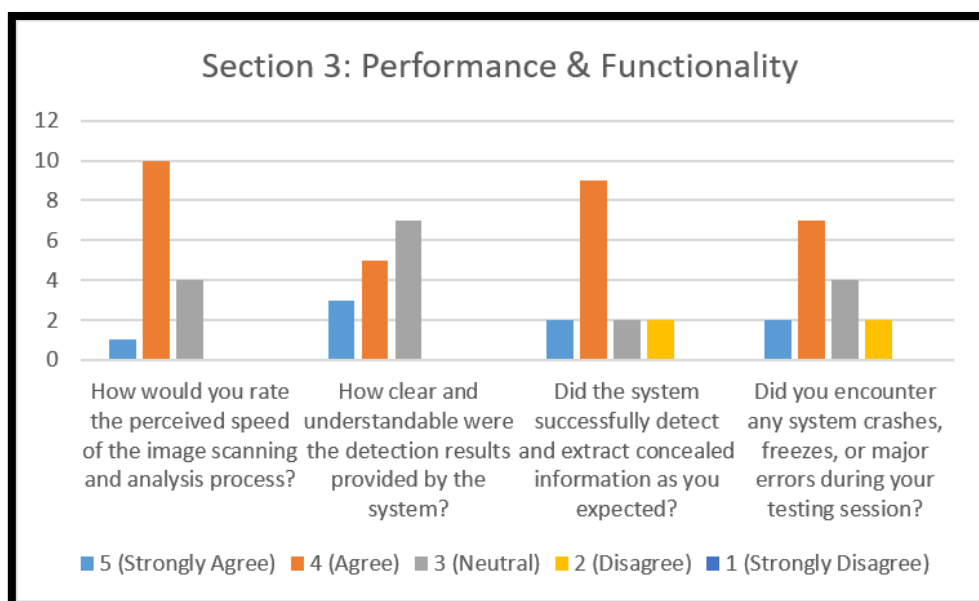


Figure 38: Performance & Functionality

5.3 Chapter Summary

This chapter is about testing the StegDetect system. The StegDetect system needs to be tested to make sure it works the way it should and that it is safe from problems. The people who made the StegDetect system did a lot of testing to make sure it works right. They tested each part of the system. Then they tested all the parts together. The StegDetect system worked well when they tested it. It let users log in safely. It could handle pictures and work with web browsers. They also tested the StegDetect system to see if it could stop people from hacking into it. The StegDetect system did a job of stopping hackers. It could stop them from guessing passwords. It could stop them from uploading bad files. The StegDetect system could even stop hackers who tried to trick it. It did not crash when it got information. Finally, they tested the StegDetect system with users. The users liked the StegDetect system. They could use it easily. The users could find information, in pictures and videos. This is what the StegDetect system is supposed to do.

CHAPTER 6: CONCLUSIONS

6.1 Introduction

This chapter talks about the end of our project. We look at what we achieved and give some advice for anyone who wants to improve our system in the future. We have some ideas for making it better.

6.2 Conclusion of the Overall Project

The StegDetect web-based system was built to tackle a problem in digital forensics. Traditional tools can't extract data even after finding it. This project used the V-Model method to create a web application with three layers. The final version lets forensic analysts and cybersecurity pros upload image files and check them using detection engines. By combining detection and extraction capabilities this project fills a gap in investigations. It provides evidence like malware or hidden messages.

6.3 Discussion of Findings

The testing phase had positive results that match the initial problem and research gaps.

- **User Acceptance Testing (UAT):** Most users (students, IT and cybersecurity pros) liked the system. Over 70% said the interface and navigation were "Very Easy" or "Easy" to use. The dashboard design, file upload and confidence score gauge helped with work. Also, over 80% found the detection and extraction capabilities effective. They liked being able to download files, like.txt and.zip.
- **Bridging the Research Gap:** As Bin et al. (2024) Said, most tools only detect data but don't extract it. StegDetect extracts hidden files without noise. This solves the problem of recovering messages.
- **Security Resilience:** Security testing worked well. The system locked out accounts after five login attempts. The multi-engine detection (using LSB, Statistical and Machine Learning methods) flagged stego-images with noise. The system handled corrupted files without crashing, keeping it stable. The StegDetect system was successful. The StegDetect project achieved its goals. The StegDetect tool is effective.

6.3 Achievement of Objectives

The project did what it was supposed to do. We have proof of that in the report.

Here are the things the project was trying to do:

- Objective 1: To study techniques used to detect and extract hidden data in multimedia files using steganalysis.

Evidence of Achievement: This in Chapter 2 by reading a lot of papers and books on the subject. We looked at how people hide data in pictures and videos and how we can find that data using tools and methods like looking at the spatial domain and the transform domain. We also used some ways of finding hidden data like looking for things that do not seem right and using special computer programs that can learn and get better at finding hidden data.

- Objective 2: To develop a web application that is able to analyse multimedia files and detect the presence of hidden data.

Evidence of Achievement: This in Chapters 3 and 4. We used Python to make the application. We used a special way of building it that has three parts: one part that people can see and use one part that does all the work and one part that stores all the data. We also made an engine that can find hidden data and it uses some special methods like looking at the way the data is spread out and using special computer programs that can learn and get better at finding hidden data.

- Objective 3: To test the application accuracy in detecting and extracting hidden data.

Evidence of Achievement: This in Chapter 5. We tested the application in different ways like making sure it works correctly making sure it is safe and making sure people can use it. One of our tests showed that the application can really find data and get it out of the files and it can even tell us what kind of file the hidden data is, in.

6.4 Limitations of the Project

The StegDetect prototype was successful. Some limitations were found during its development and testing.

- **Format Restrictions:** The StegDetect system can only work with image files like PNG, JPG and BMP. It cannot work with audio or video files.
- **Evasion by Advanced Algorithms:** The system may have a time finding hidden data that was put in using advanced algorithms that the system was not trained on.
- **Dependency on File Characteristics:** How well the system works depends on the file it is looking at. For example the image resolution and how much the file has been compressed can affect the results.
- **Scope of Steganography:** The StegDetect system only looks at media and does not look at other ways that people can hide messages like hiding them in network traffic or text.

6.5 Recommendations

Based on what we found out about the StegDetect system we have some ideas for how to make it better.

Here are some recommendations:

- **Expansion of Supported Multimedia:** The StegDetect system should be able to work with files like WAV and MP3 and video files like MP4. This would make it a powerful tool for looking at all kinds of media.
- **Integration of Advanced Deep Learning:** To find data that was put in using very advanced algorithms the StegDetect system should use something called Convolutional Neural Networks. These networks can be trained to recognize the signs that are left by modern algorithms that hide data.
- **Cloud-Based Deployment:** The StegDetect system should be moved to a cloud infrastructure like AWS or Microsoft Azure. This would give it the power to look at a lot of high-resolution images quickly and would make it easier for law enforcement agencies all over the world to use it.

REFERENCES

Alshumrani, A., Clarke, N., & Ghita, B. (2023). A Unified Forensics Analysis Approach to Digital Investigation. *International Conference on Cyber Warfare and Security*, 18(1), 466–475. <https://doi.org/10.34190/iccws.18.1.972>

Benadjila, R., Khati, L., & Vergnaud, D. (2022). Secure storage—Confidentiality and authentication. *Computer Science Review*, 44, 100465. <https://doi.org/10.1016/j.cosrev.2022.100465>

F. Abbas, N. A., Abdulredha, N., Ibrahim, R. K., & Ali, A. H. (2022). Security and imperceptibility improving of image steganography using pixel allocation and random function techniques. *International Journal of Electrical and Computer Engineering (IJECE)*, 12(1), 694. <https://doi.org/10.11591/ijece.v12i1.pp694-705>

Hildebrandt, M. (2020). On digitized forensics : novel acquisition and analysis techniques for latent fingerprints based on signal porcessing and pattern recognition. Nbn-Resolving.org. <https://opendata.uni-halle.de//handle/1981185920/35085>

Kuznetsov, O., Kostenko, O., Klymenko, K., Hbur, Z., & Kovalskyi, R. (2025). Machine Learning Analytics for Blockchain-Based Financial Markets: A Confidence-Threshold Framework for Cryptocurrency Price Direction Prediction. *Applied Sciences*, 15(20), 11145. <https://doi.org/10.3390/app152011145>

Qin, Y., Wu, H., Zhang, X., & Feng, G. (2022). Semi-Supervised Structured Subspace Learning for Multi-View Clustering. *IEEE Transactions on Image Processing*, 31, 1–14. <https://doi.org/10.1109/tip.2021.3128325>

APPENDIX



A WEB BASED STEGANALYSIS DETECTING AND EXTRACTING HIDDEN DATA IN IMAGE FILES



RESEARCHER
SUPERVISOR
GITHUB LINK

MUHAMMAD AMEER BIN IDRIS
MDM.NORHAIZA BINTI YA ABDULLAH
<https://github.com/anomalizov-max/steganalysis-fyp>

ameer.idris01@s.unikl.edu.my
norhaizaya@unikl.edu.my

0137163550
0133337700

INTRODUCTION

This project investigates and creates a web-based steganalysis system designed to recognize suspicious activity and retrieve hidden payloads from image files. The system ensures easier integration into forensic workflows and secure execution.

PROBLEM STATEMENT

- Steganalysis Tools Focus on Detection but Not Extraction
- Ineffective Payload Recovery During Investigations
- Multimedia Files Are a Security Blind Spot

OBJECTIVES

- To study techniques used to detect and extract hidden data in multimedia files using steganalysis.
- To develop a web application that is able to analyse multimedia files and detect the presence of hidden data.
- To test the application accuracy in detecting and extracting hidden data.

SCOPE

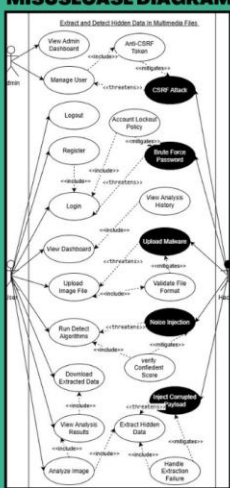
- Platform: The system is built as a web-based forensic analysis tool, making it highly accessible through any standard web browser.
- Target Users: Designed primarily for digital forensic investigators, cybersecurity analysts, law enforcement personnel, and security researchers.
- Core Functionality: Provides capabilities to upload multimedia files, analyze them for steganographic manipulation, detect the presence of hidden data, and extract the embedded payloads.
- Supported File Types: The current prototype focuses specifically on image files, supporting formats such as PNG, JPG, and BMP.
- Security Compliance: Developed in accordance with OWASP Application Security Guidelines to ensure protection against common vulnerabilities.

LITERATURE REVIEW

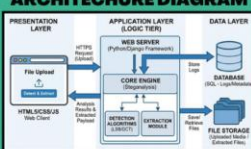
Cited Reference	The Problem	Project Improvement	Project Motivation
Hindabrunn, M. (2020). On digitized forensics: novel analysis techniques for latent fingerprints based on signal processing and pattern recognition. https://www.researchgate.net/publication/341880700	Standardized evaluation schemes are lacking to ensure the reliability and accuracy of digital forensic tools.	Implementation of the V-model methodology to provide a strict, phase-by-phase testing structure for extraction.	To ensure high quality and reliability in payload extraction, specifically to meet legal standards for forensic evidence.
Ahmed, A., Clarke, N., & Goh, S. (2023). A Unified Forensic Analysis Approach to Digital Investigation: Integrating Cyber Forensics, Incident Response, and Security. https://doi.org/10.24390/icsw.14.1.92	Inaccessible "Silent" Tools: Advanced steganographic techniques often cater only to experienced users, leaving a gap for normal investigators to detect hidden data.	Web-Based GUI: I developed a user-friendly Web Dashboard that allows users to upload and analyze files without needing coding knowledge.	To bridge the gap between "Academic Research" and "Real-world Forensics," making advanced analysis accessible to law enforcement.
Subramanian, N., Elavarasu, D., Al-Masoud, S., & Bourdoux, A. (2021). Image Steganography: A Review of the recent advances. https://doi.org/10.1109/ACCESS.2021.3053999	Perceptual Blind Spots: Human eyes and standard steganalysis tools often miss hidden data because it is hidden in the "invisible" data (noise) of high-resolution files.	Algorithmic Inspection: The system uses specific LSB & DCT Algorithms to mathematically scan these "silent spots" that human perception cannot see.	To demonstrate that "Visual Inspection is Insufficient" and that automated algorithmic scanning is required to find modern hidden threats.
Amalavanan, A. & Chandrasekar, R. (2020). A Bayesian Image Steganalysis Approach to estimate the embedded secret message. In <i>Proceedings of the 7th International Security Workshop 2020</i> , pp. 35-40. https://doi.org/10.1109/ISW45707.2020.9351145	Difficulty in Message Extraction: Traditional detection often only identifies if data exists but cannot accurately estimate the length or size of the secret message without knowing the	Probabilistic Estimation: Implementation of a Bayesian-based scoring system that uses probability distributions to estimate the likely size and presence of hidden data.	To enhance forensic precision by providing a statistical basis for "How much" data is hidden, rather than just "IF" data is hidden.
Kamran, O., Karaman, O., Myrskovic, K., Hrb, Z., & Kovalev, R. (2023). Machine Learning Analytics for Blockchain-Based Financial Markets: A Confidence-Threshold Framework for Cryptocurrency Price Direction Prediction. <i>Applied Sciences</i> , 15(20), 11145. https://doi.org/10.3390/app152011145	Binary 0/1 Thresholds in automated detection systems can lead to misclassification and missed evidence.	Integration of a Confidence Score mechanism that triggers an automated extraction module when a threshold is met.	To provide concrete evidence (timestamps, stolen credentials) for attack chain reconstruction instead of simple alerts.

SYSTEM REQUIREMENT & DESIGN

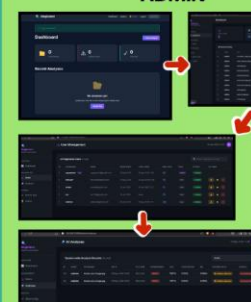
MISUSECASE DIAGRAM



ARCHITECTURE DIAGRAM



ADMIN



RESULTS & DISCUSSIONS

FUNCTIONALITY TESTING

No	Module	Actual Result	Expected Result	Result (Pass/Fail)
1	User Registration	User information has been securely recorded in the SQLite database.	The registration is successful and saved in the database.	Pass
2	User Login	User is authenticated and redirected to the main dashboard.	System grants access only registered users with correct credentials.	Pass
3	Image Upload	The image is successfully uploaded, and a secure filename is generated and stored in the server.	Image is uploaded without errors and restricted to allowed extensions (PNG, JPG, BMP).	Pass
4	Steganalysis Detection	The system processes the image and returns LSB, Statistical, and Machine Learning confidence scores.	The system accurately detects the probability of hidden data in the uploaded image.	Pass
5	Data Extraction	The system successfully extracts hidden payloads in .txt, .jpg, and .xlsx formats and returns them to the user's browser.	Hidden files are extracted with their original file types, and no garbage binary noise is produced.	Pass

INTEGRATION TESTING

No	Module	Description	Expected Result	Browser Version (Result)		
				Google Chrome	Mozilla Firefox	Brave Browser
1	System UI Rendering	The system's front-end code successfully integrates with the browser engine.	Able to view the dashboard and interact with the system application smoothly.	Pass	Pass	Pass
2	File Upload Handling	System can process and upload large image files from the client's local machine to the server.	Upload process is successful without browser timeouts.	Pass	Pass	Pass
3	Downloading Extracted Data	Server sends extracted binary files back to the user's browser.	User successfully receives the extracted hidden files via browser download manager.	Pass	Pass	Pass

SECURITY TESTING

Threat Scenario	Testing Method	Security Control	Result
Brute Force Attack	Attempting 5 times incorrect password entries	Account Lockout Policy triggers after the 5th attempt	PASS
Malware Upload	Attempting to upload .exe, .bat, or .js files	File Format Validation restricts uploads to PNG, JPG, and BMP	PASS
Noise Injection	Uploading step-images modified with random pixel noise	Multi-Engine Detection (LSB, Statistical, and ML)	PASS
Corrupted Payload	Uploading images with manually corrupted hidden bytes	Graceful Error Handling prevents system crashes during extraction	PASS

CONCLUSION

Conclusion: The project successfully delivers a robust, user-friendly forensic tool that simplifies the complex process of steganalysis, providing security professionals with actionable insights and immediately extracted payloads.

RECOMMENDATION

- Support for additional file formats including Audio (WAV/MP3) and Video.
- Advanced decryption capabilities for password-protected stego payloads.

COMMERCIALIZATION

Commercial Value: The solution is highly valuable for Digital Forensics and Incident Response (DFIR) teams, cybersecurity firms, and law enforcement agencies. It can be licensed as a cloud-based SaaS platform or deployed as an on-premise enterprise appliance for automated threat hunting and malware analysis pipelines.

USER

PROTOTYPE DEVELOPMENT

- Features of the prototype:
- Multi-Method Detection
 - Automated Data Extraction
 - Secure Web Dashboard
 - Analysis History & Logs
 - Multi-Format Support